

TechTribers

Java Interview Series · Part 17 · Q800–Q849

JDBC: Connections, Transactions · Performance

From the wire protocol to HikariCP: drivers and DataSources, statements and java.time mapping, streaming result sets, isolation and locking, batching and pool sizing, virtual threads, SQLException, testing, and where JdbcClient, JPA and jOOQ fit.

50	50	14	17–25
questions	code examples	diagrams	Java LTS

techtriber.com

@techtribers

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `JD-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**
A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**
What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**
The next question the interviewer is likely to ask, with the answer.

50	6	50	14
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🔌	01	JDBC Architecture, Drivers and Connections	Q800–Q807	5
📝	02	Statements, Parameters and Data Types	Q808–Q816	14
📑	03	ResultSet: Cursors, Streaming and Mapping	Q817–Q823	25
🔒	04	Transactions, Isolation and Locking	Q824–Q832	33
🚀	05	Batching, Connection Pooling and Performance	Q833–Q841	44
🧰	06	Errors, Security, Testing and the JDBC Ecosystem	Q842–Q849	55

© TechTribers. For personal use only. Do not redistribute.

All questions

🔌 JDBC Architecture, Drivers and Connections

Q800 What happens end to end when you run a query through JDBC, from `DataSource` to the database wire protocol? 5

Q801 What are the four JDBC driver types, and which one do you use today? 6

Q802 How are JDBC drivers loaded today, and why is `Class.forName("...Driver")` no longer needed? 7

Q803 `DriverManager` vs `DataSource`: why should production code use a `DataSource`? 8

Q804 What goes into a JDBC URL, and which connection properties matter in production? 10

Q805 Why is a JDBC `Connection` expensive, and can it be shared between threads? 11

Q806 How do you close JDBC resources correctly, and what does closing a `Connection` close? 12

Q807 Which JDBC 4.1, 4.2 and 4.3 features should a senior developer know? 13

📝 Statements, Parameters and Data Types

Q808 When do you use `CallableStatement`, and how do you read OUT parameters? 14

Q809 How exactly does `PreparedStatement` prevent SQL injection, and what can it not parameterize? 15

Q810 How do you bind an `IN` clause with a variable number of values? 17

Q811 What are server-side prepared statements and driver statement caching? 18

Q812 `execute` vs `executeQuery` vs `executeUpdate` vs `executeLargeUpdate`: when do you use which? 19

Q813 How do you retrieve auto-generated keys after an insert? 20

Q814 How do you handle NULL values in JDBC (`setNull`, `wasNull`, `getObject` with a type)? 21

Q815 How do you map `java.time` types with JDBC and avoid time-zone bugs? 22

Q816 How do you read and write BLOB and CLOB data efficiently? 23

📑 ResultSet: Cursors, Streaming and Mapping

Q817 How does a `ResultSet` cursor work, and what are the result set types and concurrency modes? 25

Q818 What is fetch size, and how do you stream millions of rows without running out of memory? 26

Q819 What are `ResultSetMetaData` and `DatabaseMetaData` used for? 28

Q820 How do you map `ResultSet` rows to records cleanly with a RowMapper? 29

Q821 Column index vs column label, and why does `getInt` return `0` for SQL NULL? 30

Q822 What are updatable result sets, and why are they rarely used? 31

Q823 How do query timeouts, `setMaxRows` and `Statement.cancel` work? 32

🔒 Transactions, Isolation and Locking

Q824 What are the most common JDBC transaction bugs, and what does a correct transaction template look like? 33

Q825 What are savepoints, and when are they useful? 34

Q826 Which isolation levels does JDBC define, and which anomalies does each prevent? 35

Q827 What isolation level does each major database use by default, and how does MVCC change the picture? 37

Q828 Optimistic vs pessimistic locking with plain JDBC (version column, `SELECT ... FOR UPDATE`, `SKIP LOCKED`) 38

Q829 How do you handle deadlocks and serialization failures with retries? 39

Q830 What does `setReadOnly(true)` do, and how do you route reads to replicas? 40

Q831 What are XA distributed transactions, and why do microservices usually avoid them? 41

Q832 How does Spring's `@Transactional` map onto JDBC connections? 42

© TechTribers. For personal use only. Do not redistribute.

🚀 Batching, Connection Pooling and Performance

Q833 How does JDBC batch processing work, and how large should a batch be? 44

Q834 What do `rewriteBatchedStatements` (MySQL) and `reWriteBatchedInserts` (PostgreSQL) do? 46

Q835 How do you handle `BatchUpdateException` and partial batch failures? 47

Q836 What is connection pooling, and how does HikariCP work internally? 48

Q837 How do you size a database connection pool? 49

Q838 Which HikariCP settings matter in production? 50

Q839 How do you detect and fix connection leaks? 51

Q840 How do you spot and fix the N+1 query problem at the JDBC level? 52

Q841 How do virtual threads interact with JDBC drivers and connection pools? 53

🧰 Errors, Security, Testing and the JDBC Ecosystem

Q842 How is `SQLException` structured (SQLState, vendor code, chaining, transient vs non-transient)? 55

Q843 What is `SQLWarning`, and when should you read it? 56

Q844 How do you secure JDBC access in production? 57

Q845 How do you test JDBC code reliably (Testcontainers vs H2)? 58

Q846 What do Spring's `JdbcTemplate` and `JdbcClient` remove compared with raw JDBC? 59

Q847 JDBC vs JPA vs jOOQ vs MyBatis: when do you choose which? 61

Q848 What is R2DBC, and do virtual threads make it unnecessary? 62

Q849 Coding: write a small reusable JDBC helper with query, update and transaction support. 63

© TechTribers. For personal use only. Do not redistribute.

Section 01

JDBC Architecture, Drivers and Connections

What really happens between your code and the database: driver types and loading, DataSource versus DriverManager, URLs and production connection properties, why connections are expensive, closing resources correctly, and the JDBC 4.2/4.3 features worth knowing.

Questions 800–807 · 8 questions



Q800 JD-001

• Intermediate

What happens end to end when you run a query through JDBC, from `DataSource` to the database wire protocol?

⚡ Quick answer

Your code asks a `DataSource` (usually a pool such as HikariCP) for a `Connection`; the pool hands out an already-open physical connection wrapped in a proxy. You create a `PreparedStatement`, bind parameters and execute: the **driver** encodes the SQL and parameters into the database's **wire protocol** (PostgreSQL frontend/backend protocol, MySQL protocol, Oracle Net/TTC) and sends it over TCP. The server parses, plans and executes, then streams rows back; the driver decodes them into a `ResultSet`, buffering according to the **fetch size**. Closing the connection returns it to the pool rather than closing the socket.

Where the time goes in a typical request: borrowing from a warm pool costs microseconds; the **network round trip** costs 0.2–1 ms inside a data centre (much more across regions); parsing and planning cost something on the first execution and less once the statement is cached; and transferring and decoding rows costs time proportional to the rows and columns you fetch.

That breakdown explains most JDBC tuning advice: reuse connections (pooling), reuse plans (prepared statements), cut round trips (batching, joins instead of N+1, sensible fetch size), and select only the columns you need.

JDBC itself is only an **API** (`java.sql`, `javax.sql`) plus a service-provider contract. Everything that talks to the database lives in the vendor's driver JAR.

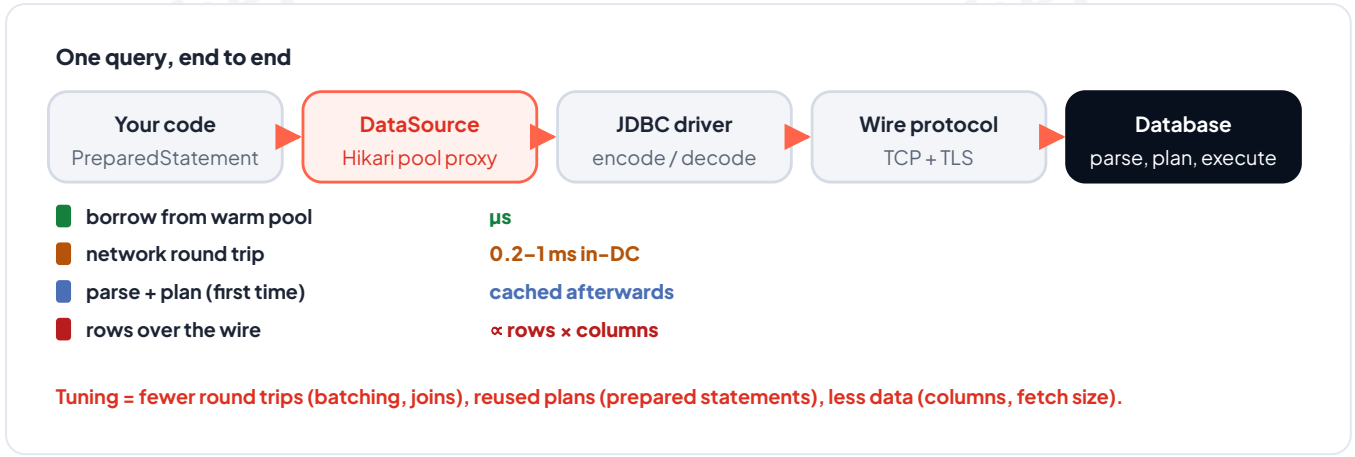
Java

```
@Autowired DataSource dataSource;                    // HikariDataSource in Spring Boot

List<Order> recentOrders(long customerId) throws SQLException {
    String sql = """
        SELECT id, total, created_at
        FROM orders
        WHERE customer_id = ?
```


© TechTribers. For personal use only. Do not redistribute.

```
        ORDER BY created_at DESC
        LIMIT 20
        """;
    try (Connection con = dataSource.getConnection();    // borrowed from the pool
         PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setLong(1, customerId);                       // bound, never concatenated
        try (ResultSet rs = ps.executeQuery()) {         // one network round trip
            List<Order> out = new ArrayList<>();
            while (rs.next()) {
                out.add(new Order(rs.getLong("id"),
                                  rs.getBigDecimal("total"),
                                  rs.getObject("created_at", OffsetDateTime.class)));
            }
            return out;
        }
    }   // the connection goes back to the pool
}
```



💡 Pro tip

When you debug latency, separate the layers: pool wait time (Hikari's `pool.Wait` metric), statement execution time on the server (`pg_stat_statements`, MySQL's slow log) and time spent iterating the `ResultSet` in Java. They point to different fixes.

Q801 JD-002 • Basic

What are the four JDBC driver types, and which one do you use today?

⚡ Quick answer

Type 1: JDBC–ODBC bridge (removed in Java 8). **Type 2**: native-API driver that calls a vendor C library (Oracle OCI). **Type 3**: network-protocol driver talking to a middleware server that talks to the database. **Type 4**: pure-Java driver speaking the database's own wire protocol directly. Practically every modern driver is **Type 4**: PostgreSQL `pgjdbc`, MySQL Connector/J, MariaDB, Oracle Thin, SQL Server `mssql-jdbc`, H2.

© TechTribers. For personal use only. Do not redistribute.

Why Type 4 won: no native code to install on every server or container image, no extra network hop, it runs anywhere a JVM runs, and it can be upgraded like any other dependency.

Where you still meet the others: **Oracle OCI (Type 2)** for features that need the native client, such as some TAF configurations or local IPC; **Type 3**-style setups appear as proxies in front of databases (connection routers, Google Cloud SQL Auth Proxy, AWS RDS Proxy), though there the application still uses a normal Type 4 driver talking to the proxy.

Logging the driver and server version at startup, as above, saves time in incident reviews: many "database bugs" turn out to be an old driver.

```java
// Type 4 drivers: one JAR on the classpath, no native libraries, no middleware
// Maven coordinates (versions omitted)
//   org.postgresql:postgresql
//   com.mysql:mysql-connector-j
//   com.oracle.database.jdbc:ojdbc11
//   com.microsoft.sqlserver:mssql-jdbc

try (Connection con = dataSource.getConnection()) {
    DatabaseMetaData md = con.getMetaData();
    log.info("driver {} {} | JDBC {}.{} | {} {}",
            md.getDriverName(), md.getDriverVersion(),
            md.getJDBCMajorVersion(), md.getJDBCMinorVersion(),
            md.getDatabaseProductName(), md.getDatabaseProductVersion());
}
```

🎯 **Likely follow-up**

Which JDBC version does your driver implement? `getJDBCMajorVersion()` / `getJDBCMinorVersion()` report it; current drivers implement JDBC 4.2 or 4.3.

🚚 **Q802** JD-003 • Intermediate

How are JDBC drivers loaded today, and why is `Class.forName("...Driver")` no longer needed?

⚡ **Quick answer**

Since **JDBC 4.0 (Java 6)**, every driver JAR contains `META-INF/services/java.sql.Driver` naming its driver class, and `DriverManager` discovers drivers with `ServiceLoader` the first time it's used. Each driver registers itself in a static initializer. `Class.forName("org.postgresql.Driver")` was the pre-4.0 way to force that static initializer to run; with a JDBC 4 driver on the class path it's redundant. It can still matter in unusual class-loader setups, where `DriverManager` can't see a driver loaded by a child loader.

How `DriverManager.getConnection(url, ...)` picks a driver: it asks each registered driver `acceptsURL(url)` and uses the first one that accepts it. A `No suitable driver found for jdbc:...` error therefore means one of three things: the driver JAR isn't on the class path, the URL prefix is wrong (`jdbc:postgres:` instead of `jdbc:postgresql:`), or the driver was loaded by a class loader `DriverManager` doesn't trust.

© TechTribers. For personal use only. Do not redistribute.

The class-loader case shows up in app servers and plugin systems: `DriverManager` only returns drivers visible to the caller's class loader. Using a `DataSource` created by the application (the normal case with connection pools) side-steps this entirely, because the pool instantiates the driver class itself.

```java
// Old (JDBC 3 era) - still compiles, no longer necessary
Class.forName("org.postgresql.Driver");

// Today: the driver is found through META-INF/services/java.sql.Driver
try (Connection con = DriverManager.getConnection(
        "jdbc:postgresql://db:5432/shop", "app", secret)) {
    // ...
}

// Which drivers did DriverManager discover?
DriverManager.drivers()                 // Java 9+: Stream<Driver>
        .map(d -> d.getClass().getName())
        .forEach(System.out::println);
```

⚠️ **Interview trap**

Leaving `Class.forName(...)` with a hard-coded driver class in shared code. It adds nothing and breaks when you switch databases or when the driver class is renamed (MySQL's `com.mysql.jdbc.Driver` became `com.mysql.cj.jdbc.Driver` in Connector/J 8).

Q803 JD-004 • Intermediate

`DriverManager` vs `DataSource`: why should production code use a `DataSource`?

⚡ **Quick answer**

`DriverManager.getConnection()` opens a **new physical connection** on every call (TCP, TLS, authentication, session setup: often several milliseconds) and gives you no pooling, no central configuration and no monitoring. A `DataSource` is an interface for a connection factory that can be configured once and injected: a **pooled** implementation such as HikariCP reuses connections, enforces limits and timeouts, validates connections and exposes metrics. Frameworks (Spring, Jakarta EE, Micronaut, Quarkus) all work in terms of `DataSource`.

What a `DataSource` gives you beyond pooling:

- **Decoupling**: code depends on `javax.sql.DataSource`, so tests can inject an embedded or Testcontainers database and production can inject a pool, a routing data source or a proxy (for example a tracing wrapper).
- **Configuration in one place**: credentials, URL, timeouts, TLS settings.
- **Specialised variants**: `ConnectionPoolDataSource` and `XADataSource` let pools and transaction managers manage physical and XA connections.

`DriverManager` is still fine for command-line tools, migrations run once, and small examples.

© TechTribers. For personal use only. Do not redistribute.

```java
// Plain JDBC, no framework: build the pool once at startup
HikariConfig cfg = new HikariConfig();
cfg.setJdbcUrl("jdbc:postgresql://db:5432/shop");
cfg.setUsername("app");
cfg.setPassword(System.getenv("DB_PASSWORD"));
cfg.setMaximumPoolSize(10);
cfg.setPoolName("shop-db");
DataSource ds = new HikariDataSource(cfg);       // close it on shutdown

// Everywhere else: depend on the interface, not on the pool
final class OrderRepository {
    private final DataSource ds;
    OrderRepository(DataSource ds) { this.ds = ds; }
}

// Spring Boot: the same pool from application.properties
//   spring.datasource.url=jdbc:postgresql://db:5432/shop
//   spring.datasource.hikari.maximum-pool-size=10
```

DriverManager vs pooled DataSource

`DriverManager.getConnection()`

- TCP connect
- TLS handshake
- authenticate
- session setup (PG: new backend process)

✗ … every single call: several ms

`pool.getConnection()`



✓ hand out an open connection: µs

✓ close() = return to pool

✓ limits, timeouts, metrics, validation

One pool per database per application, created once and injected as javax.sql.DataSource.

⚠️ **Interview trap**

Creating a new `HikariDataSource` per request or per repository instance. Each one is a separate pool with its own connections and background threads, and you'll exhaust the database's connection limit quickly. One pool per database per application, shared.

© TechTribers. For personal use only. Do not redistribute.

Q804 JD-005

Intermediate

What goes into a JDBC URL, and which connection properties matter in production?

⚡ **Quick answer**

A JDBC URL is `jdbc:<subprotocol>:<driver-specific part>`, typically host, port, database and `key=value` properties. The properties that matter most in production are: **timeouts** (connect, socket/read), **TLS** settings (`sslmode=verify-full` and similar), **application name** (so DBAs can see who is connected), **default schema**, **prepared-statement behaviour**, **batch rewriting**, and for clusters, **multiple hosts and target server type** for failover.

Every driver uses its own property names and units (pgjdbc timeouts are in seconds, Connector/J in milliseconds), so always check the driver docs rather than copying properties between databases.

Why the timeouts matter: without a **socket timeout**, a thread can block forever on a half-open TCP connection after a network partition or a database failover; with a pool of 10 connections that is enough to stall the whole service. Set the socket timeout above your longest legitimate query, and use `Statement.setQueryTimeout` (Q823) for per-query limits.

Keep credentials **out of the URL**: pass them as properties from a secret store, because URLs end up in logs, error messages and thread dumps.

Terminal

```
# PostgreSQL (pgjdbc)
jdbc:postgresql://db1:5432,db2:5432/shop
    ?targetServerType=primary          # failover across listed hosts
    &connectTimeout=5                  # seconds
    &socketTimeout=30                  # seconds; 0 = wait forever
    &sslmode=verify-full
    &ApplicationName=order-service
    &reWriteBatchedInserts=true

# MySQL (Connector/J)
jdbc:mysql://db:3306/shop
    ?connectTimeout=5000&socketTimeout=30000   # milliseconds
    &sslMode=VERIFY_IDENTITY
    &rewriteBatchedStatements=true
    &cachePrepStmts=true&useServerPrepStmts=true

# SQL Server
jdbc:sqlserver://db:1433;databaseName=shop;encrypt=true;loginTimeout=5
```

💡 **Pro tip**

Set `ApplicationName` (PostgreSQL) or `connectionAttributes` (MySQL) to the service name. During an incident `pg_stat_activity` then shows which service holds the blocking connections.

© TechTribers. For personal use only. Do not redistribute.

Q805 JD-006

• Intermediate

Why is a JDBC `Connection` expensive, and can it be shared between threads?

> ⚡ **Quick answer**
>
> Opening a physical connection costs a TCP handshake, usually a TLS handshake, authentication, and server-side session setup (on PostgreSQL, a whole **backend process** per connection; on MySQL a thread and per-session buffers). That's typically several milliseconds and real server memory. A `Connection` also carries **session state**: the current transaction, auto-commit, isolation level, schema, temporary tables. So: **don't share one connection between concurrent threads**. Borrow one per unit of work from a pool, use it on one thread, and return it.

The JDBC spec asks drivers to be thread-safe in the narrow sense that concurrent calls won't corrupt the driver. That doesn't make sharing useful: a connection executes **one statement at a time** on the wire, so threads serialize on it, and they silently share each other's transactions.

Server-side cost is why pools are small. A PostgreSQL server with a few hundred connections spends noticeable memory and CPU on them even when idle; tools such as **PgBouncer** exist to multiplex many client connections onto fewer server connections.

With **virtual threads** the rule is unchanged: thousands of virtual threads can run, but they still borrow from the same small pool (Q841).

Java

```java
// WRONG: one shared connection for the whole application
static final Connection SHARED = DriverManager.getConnection(url, user, pass);
// thread A: SHARED.setAutoCommit(false); ... thread B's statements join A's transaction
// thread B: SHARED.commit();                ... commits A's half-finished work

// RIGHT: one connection per unit of work, from a pool
void transfer(long from, long to, BigDecimal amount) throws SQLException {
    try (Connection con = dataSource.getConnection()) {
        con.setAutoCommit(false);
        try {
            debit(con, from, amount);
            credit(con, to, amount);
            con.commit();
        } catch (SQLException | RuntimeException e) {
            con.rollback();
            throw e;
        }
    }
}
```

> ⚠️ **Interview trap**
>
> Holding a connection open across slow non-database work (an HTTP call, a message publish) inside a transaction. The connection, and any row locks, are tied up for the whole duration. Fetch, close, call the service, then open a new short transaction if needed.

© TechTribers. For personal use only. Do not redistribute.

Q806 JD-007

• Basic

How do you close JDBC resources correctly, and what does closing a `Connection` close?

⚡ **Quick answer**

Use **try-with-resources** for `Connection`, `Statement` / `PreparedStatement` and `ResultSet`; they are closed in reverse order even when an exception is thrown, and exceptions from `close()` are attached as **suppressed** exceptions instead of hiding the original. By the spec, closing a `Statement` closes its current `ResultSet`, and closing a `Connection` should release its statements. With a pool, `close()` on the connection **returns it to the pool**; the pool resets state, but don't rely on that to clean up open statements and result sets.

Details worth knowing:

- `Statement.closeOnCompletion()` (JDBC 4.1) closes the statement automatically when its last result set is closed. It's useful when a method returns a `ResultSet` and the caller only closes that.
- Re-executing a statement **closes its previous** `ResultSet`. Iterating two result sets from the same statement at once doesn't work.
- Leaked `ResultSet`s keep **server-side cursors** open (Oracle's `ORA-01000: maximum open cursors exceeded` is the classic symptom).
- Leaked connections drain the pool. HikariCP's `leakDetectionThreshold` logs the stack trace of the code that borrowed a connection and kept it too long (Q839).

Java

```java
try (Connection con = ds.getConnection();
     PreparedStatement ps = con.prepareStatement(
             "SELECT id, name FROM customer WHERE city = ?")) {
    ps.setString(1, city);
    try (ResultSet rs = ps.executeQuery()) {      // separate block: needs the binding first
        while (rs.next()) {
            consume(rs.getLong(1), rs.getString(2));
        }
    }
}   // closes rs, then ps, then con (returned to the pool)

// Pre-Java-7 style you will still see in old code: easy to get wrong
Connection con = null;
try {
    con = ds.getConnection();
    // ...
} finally {
    if (con != null) try { con.close(); } catch (SQLException ignore) { }
}
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Returning a `ResultSet` from a method whose try-with-resources has already closed the statement or connection. The caller gets `SQLException: This ResultSet is closed`. Map rows to objects inside the method and return the objects.

🆕 **Q807** `JD-008` • Advanced

Which JDBC 4.1, 4.2 and 4.3 features should a senior developer know?

⚡ **Quick answer**
JDBC 4.1 (Java 7): `AutoCloseable` resources, `getObject(column, Class<T>)`, `Connection.setSchema`, `abort` and `setNetworkTimeout`. **JDBC 4.2 (Java 8)**: `java.time` support through `setObject` / `getObject` (`LocalDate`, `LocalDateTime`, `OffsetDateTime`), `executeLargeUpdate` and other `long` counters, `SQLType` / `JDBCType`, `REF_CURSOR`. **JDBC 4.3 (Java 9)**: `Statement.enquoteIdentifier` / `enquoteLiteral`, `beginRequest` / `endRequest` hints for pools, sharding keys, and `DriverManager.drivers()`.

Two of these change everyday code:

- `java.time` **mapping** (4.2) removes the `java.sql.Timestamp` conversions that caused most time-zone bugs (Q815). Driver support varies in detail: PostgreSQL supports `OffsetDateTime` for `timestamptz`, but not `ZonedDateTime` or `Instant` directly in `setObject` in all versions; check your driver.
- `enquoteIdentifier` (4.3) gives a vendor-correct way to quote a table or column name. It still isn't validation: combine it with an **allow-list** of permitted columns (Q809).

`beginRequest` / `endRequest` let a pool tell the driver where a logical request starts and ends; Oracle's driver uses this for application continuity and request boundaries. Most applications get it automatically from their pool, if at all.

Java

```java
// 4.2: java.time without java.sql.Date/Timestamp conversions
ps.setObject(1, LocalDate.of(2026, 10, 1));
LocalDate shipped = rs.getObject("shipped_on", LocalDate.class);
OffsetDateTime at = rs.getObject("created_at", OffsetDateTime.class);

// 4.2: row counts above Integer.MAX_VALUE
long purged = ps.executeLargeUpdate();

// 4.1: per-connection network timeout (milliseconds)
con.setNetworkTimeout(executor, 30_000);

// 4.3: safe quoting for identifiers that cannot be bound as parameters
String col = stmt.enquoteIdentifier(sortColumn, false);   // "created_at"
String sql = "SELECT id FROM orders ORDER BY " + col;
```


© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

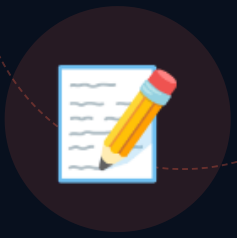
Is there a JDBC 4.4 or 5? Not as of Java 25: the JDBC API has been stable at 4.3 since Java 9, with improvements happening in drivers and pools instead.

Section 02

Statements, Parameters and Data Types

CallableStatement and OUT parameters, what PreparedStatement can and can't protect, IN lists, driver-side statement caching, the execute variants, generated keys, NULL handling, java.time mapping without time-zone bugs, and large objects.

Questions 808–816 · 9 questions



Q808 JD-009 • Intermediate

When do you use `CallableStatement`, and how do you read OUT parameters?

⚡ **Quick answer**

`CallableStatement` calls **stored procedures and functions** using the JDBC escape syntax `{call proc(?, ?)}` or `{? = call func(?)}`. Bind IN parameters with `setXxx`, declare OUT parameters with `registerOutParameter(index, type)` **before** executing, then read them with `getXxx` after `execute()`. A procedure can also return one or more result sets, which you read before the OUT parameters.

Practical notes:

- Read **all result sets first** (`getResultSet()`, `getMoreResults()`), then OUT parameters. Some drivers make OUT values available only after results are consumed.
- `Types.REF_CURSOR` (JDBC 4.2) or vendor types (`OracleTypes.CURSOR`) let a procedure return a cursor as an OUT parameter.
- Named parameters (`cs.setString("p_sku", sku)`) are optional in the spec and not supported by every driver; positional is portable.

Whether to use stored procedures at all is an architecture decision: they keep data-heavy logic close to the data and reduce round trips, but they're harder to version, test and deploy alongside application code. Many teams limit them to bulk operations and reporting.

```java
// Function returning a value: {? = call ...}
try (CallableStatement cs = con.prepareCall("{? = call order_total(?)}")) {
```


© TechTribers. For personal use only. Do not redistribute.

```
    cs.registerOutParameter(1, Types.NUMERIC);
    cs.setLong(2, orderId);
    cs.execute();
    BigDecimal total = cs.getBigDecimal(1);
}

// Procedure with IN and OUT parameters
try (CallableStatement cs = con.prepareCall("{call reserve_stock(?, ?, ?)}")) {
    cs.setString(1, sku);
    cs.setInt(2, quantity);
    cs.registerOutParameter(3, Types.INTEGER);      // remaining stock
    cs.execute();
    int remaining = cs.getInt(3);
}

// PostgreSQL 11+ procedures can also be called with plain SQL
try (CallableStatement cs = con.prepareCall("CALL archive_orders(?)")) {
    cs.setObject(1, LocalDate.now().minusYears(2));
    cs.execute();
}
```

⚠️ **Interview trap**

Calling `getInt(3)` on an OUT parameter you never registered, or registering it after `execute()`. Drivers throw an exception or return garbage; register before executing.

Q809 JD-010 • Advanced

How exactly does `PreparedStatement` prevent SQL injection, and what can it not parameterize?

⚡ **Quick answer**

With a `PreparedStatement`, the SQL text and the parameter **values travel separately**: the values are never parsed as SQL, so `' OR '1'='1` is just a strange string. (With server-side preparation the database receives them in separate protocol messages; with client-side preparation the driver escapes them correctly.) But a placeholder can only stand for a **value**: you can't bind **identifiers** (table, column names), **keywords** (`ASC` / `DESC`), `ORDER BY` columns, `LIMIT` in some databases, or a whole **IN list**. Those parts must come from an **allow-list**, never directly from user input.

Things that still go wrong with prepared statements:

- **Concatenating before preparing**: `con.prepareStatement("... WHERE name = '" + name + "'")` is a prepared statement with an injection in it.
- **Dynamic identifiers** from request parameters (`?sort=`, `?table=`) appended without an allow-list.
- `LIKE` **patterns**: binding protects against injection, but user-supplied `%` and `_` still act as wildcards; escape them if the search must be literal.
- **Stored procedures** that build dynamic SQL inside the database (`EXECUTE format(...)`) reintroduce injection on the server side.

© TechTribers. For personal use only. Do not redistribute.

`Statement.enquoteIdentifier` (JDBC 4.3) quotes an identifier correctly, but quoting isn't authorisation: a correctly quoted `"salary"` column is still one the user shouldn't sort or filter by. Allow-lists solve both problems.

Java

```java
private static final Map<String, String> SORTABLE = Map.of(
        "date", "created_at",
        "total", "total",
        "customer", "customer_name");

List<Order> search(String status, String sortKey, boolean desc) throws SQLException {
    String column = SORTABLE.get(sortKey);              // allow-list: user picks a key
    if (column == null) {
        throw new IllegalArgumentException("cannot sort by " + sortKey);
    }
    String sql = "SELECT id, total, created_at FROM orders WHERE status = ? "
               + "ORDER BY " + column + (desc ? " DESC" : " ASC");
    try (Connection con = ds.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, status);                        // value: always bound
        return mapOrders(ps.executeQuery());
    }
}
```

What a ? placeholder can and cannot stand for

```
SELECT id FROM orders WHERE status = ? AND city IN (?, ?, ?) ORDER BY created_at DESC
```

- VALUES: bind with ? → sent separately, never parsed as SQL
- IDENTIFIERS and KEYWORDS: table, column, ORDER BY, ASC/DESC → allow-list only

`Map<String,String> SORTABLE = {"date": "created_at", "total": "total"} → user picks a KEY, never raw SQL`

Still injectable: "... WHERE name = '" + name + "'" passed to prepareStatement(); dynamic SQL inside procedures.

🎯 Likely follow-up

Does an ORM make you safe? JPQL/HQL with named parameters and the Criteria API are safe in the same way; string-built JPQL and native queries are not.

© TechTribers. For personal use only. Do not redistribute.

Q810 JD-011

• Intermediate

How do you bind an `IN` clause with a variable number of values?

> ⚡ **Quick answer**
> A single `?` can't hold a list, so either **generate one placeholder per value** (`IN (?, ?, ?)`) and bind each, or use a **database array** (`= ANY(?)` with `con.createArrayOf` on PostgreSQL). Generating placeholders works everywhere but produces a different SQL string for every list size, which hurts statement caching; **padding** the list to fixed sizes (for example powers of two) limits the number of distinct statements. Very large lists should become a temporary table or a join instead.

Limits to keep in mind: Oracle rejects more than **1,000** literals in one `IN` list (`ORA-01795`); SQL Server caps a statement at about **2,100** parameters; PostgreSQL's protocol limit is **65,535** bind parameters. Long `IN` lists also make plans worse on some databases.

For thousands of IDs, better options are: batch the lookups into chunks; insert the IDs into a temporary table and join; or (PostgreSQL) pass an array and use `unnest` . Spring's `NamedParameterJdbcTemplate` expands a `Collection` parameter into the right number of placeholders for you, but it doesn't pad.

Java

```java
// Portable: one placeholder per value, padded to limit distinct SQL strings
List<Product> findByIds(List<Long> ids) throws SQLException {
    if (ids.isEmpty()) return List.of();               // "IN ()" is a syntax error
    int size = Integer.highestOneBit(ids.size() - 1) << 1;   // 1,2,4,8,16...
    size = Math.max(1, size);
    String marks = String.join(", ", Collections.nCopies(size, "?"));
    String sql = "SELECT id, name, price FROM product WHERE id IN (" + marks + ")";
    try (Connection con = ds.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {
        for (int i = 0; i < size; i++) {
            ps.setLong(i + 1, ids.get(Math.min(i, ids.size() - 1)));  // repeat the last id
        }
        return mapProducts(ps.executeQuery());
    }
}

// PostgreSQL: one placeholder, one statement for every list size
String sql = "SELECT id, name, price FROM product WHERE id = ANY(?)";
ps.setArray(1, con.createArrayOf("bigint", ids.toArray()));
```

> ⚠️ **Interview trap**
> Forgetting the empty-list case. `WHERE id IN ()` is invalid SQL on most databases; return early (or use `WHERE 1=0` deliberately).

© TechTribers. For personal use only. Do not redistribute.

Q811 JD-012 • Advanced

What are server-side prepared statements and driver statement caching?

⚡ **Quick answer**

A **server-side prepared statement** is parsed and planned once by the database and then executed many times with new parameters, saving parse and plan work. Drivers decide when to use them: **pgjdbc** switches to a named server-side statement after the same statement has been executed **5 times** (`prepareThreshold`); **MySQL Connector/J** emulates prepares on the client by default (`useServerPrepStmts=false`). Because a server statement belongs to one connection, drivers or pools also keep a **per-connection statement cache** so `prepareStatement(sameSql)` doesn't prepare again.

Why it matters only if the SQL text is stable: the cache key is the **exact SQL string**. Concatenated values (`WHERE id = 42`) create a new statement per value, defeat caching, and fill the server's plan cache. Bound parameters keep one string.

Trade-offs to know about:

- **Generic vs custom plans**: once PostgreSQL uses a prepared statement repeatedly it may switch to a *generic* plan that ignores the actual parameter values; for skewed data that can be slower (tunable with `plan_cache_mode`).
- **Poolers**: PgBouncer in *transaction* mode historically broke named server-side statements, because the next transaction may run on a different server connection. Older setups used `prepareThreshold=0` ; PgBouncer 1.21+ can track prepared statements (`max_prepared_statements`).
- **Schema changes** can invalidate cached plans; pgjdbc retries some "cached plan must not change result type" errors automatically.

Terminal

```
# PostgreSQL (pgjdbc)
prepareThreshold=5            # executions before switching to a server-side statement
preparedStatementCacheQueries=256
preparedStatementCacheSizeMiB=5

# MySQL (Connector/J)
useServerPrepStmts=true       # real server-side prepares (off by default)
cachePrepStmts=true           # keep prepared statements per connection
prepStmtCacheSize=250
prepStmtCacheSqlLimit=2048

# Oracle: implicit statement cache on the connection
oracle.jdbc.implicitStatementCacheSize=50
```

💡 **Pro tip**

HikariCP intentionally has no statement cache of its own; its documentation recommends using the driver's cache, configured with properties like those above.

© TechTribers. For personal use only. Do not redistribute.

Q812 JD-013

Basic

`execute` vs `executeQuery` vs `executeUpdate` vs `executeLargeUpdate`: when do you use which?

> ⚡ **Quick answer**
> `executeQuery()` is for statements that return a **result set** (`SELECT`) and returns a `ResultSet`. `executeUpdate()` is for **DML/DDL** (`INSERT`, `UPDATE`, `DELETE`, `CREATE`) and returns the affected row count as an `int`; `executeLargeUpdate()` (JDBC 4.2) returns a `long`. `execute()` works for **anything**, including statements whose kind you don't know or that return several results: it returns `true` if the first result is a `ResultSet`, and you walk the results with `getResultSet()`, `getUpdateCount()` and `getMoreResults()`.

Use the **row count** from `executeUpdate`; it's free information. `0` rows from an `UPDATE ... WHERE id = ? AND version = ?` is how optimistic locking detects a conflict (Q828).

Row counts are not identical across databases. The MySQL server natively reports rows **changed**, so updating a row to its current values counts `0`; Connector/J by default (`useAffectedRows=false`) asks the server for rows **matched** instead, which makes such an update count `1`. Setting `useAffectedRows=true` switches back. Write a test around this if your logic depends on the count.

Calling `executeQuery` on a statement that returns no result set throws `SQLException` in most drivers, and vice versa.

Java

```
int changed = ps.executeUpdate();                    // UPDATE ... -> rows changed
if (changed == 0) {
    throw new OptimisticLockException("order " + id + " was modified or deleted");
}

// PostgreSQL: UPDATE ... RETURNING produces a result set, so use executeQuery
try (PreparedStatement p = con.prepareStatement(
        "UPDATE account SET balance = balance - ? WHERE id = ? RETURNING balance")) {
    p.setBigDecimal(1, amount);
    p.setLong(2, accountId);
    try (ResultSet rs = p.executeQuery()) {
        BigDecimal newBalance = rs.next() ? rs.getBigDecimal(1) : null;
    }
}

// Unknown or multi-result SQL (admin tools, scripts)
boolean isResultSet = st.execute(sql);
while (true) {
    if (isResultSet) {
        try (ResultSet rs = st.getResultSet()) { print(rs); }
    } else if (st.getUpdateCount() == -1) {
        break;                                       // no more results
    } else {
        System.out.println(st.getUpdateCount() + " rows");
    }
```


© TechTribers. For personal use only. Do not redistribute.

```
    isResultSet = st.getMoreResults();
}
```

⚠️ **Interview trap**

Treating `executeUpdate()` returning `1` as proof that the row was inserted with your values. Triggers, `ON CONFLICT DO NOTHING` and `INSERT IGNORE` change the counts; check the semantics of the exact statement.

🔑 **Q813** JD-014 • Intermediate

How do you retrieve auto-generated keys after an insert?

⚡ **Quick answer**

Pass `Statement.RETURN_GENERATED_KEYS` (or the **names** of the key columns) to `prepareStatement`, execute the insert, then read `ps.getGeneratedKeys()`, which returns a `ResultSet` of the generated values. Naming the columns is more portable and avoids drivers returning every column. On PostgreSQL the same thing can be written explicitly with `INSERT ... RETURNING id`, which also works for multi-row inserts and returns any columns you want.

Database specifics:

- **PostgreSQL**: the driver appends `RETURNING` for you; with `RETURN_GENERATED_KEYS` it returns **all** columns (`RETURNING *`), so pass column names.
- **MySQL**: returns `LAST_INSERT_ID()` values for `AUTO_INCREMENT` columns.
- **Oracle**: identity columns and sequences work with column names; without them you may get a `ROWID` back.
- **SQL Server**: `IDENTITY` values via `SCOPE_IDENTITY()`; batch support for generated keys has historically been limited.

An alternative that avoids the round trip question entirely: generate IDs in the application (UUIDv7, Snowflake-style IDs, or a sequence fetched in blocks), which also makes batching and idempotent inserts easier.

Java

```
String sql = "INSERT INTO orders (customer_id, total, status) VALUES (?, ?, ?)";
try (PreparedStatement ps = con.prepareStatement(sql, new String[] {"id"})) {
    ps.setLong(1, customerId);
    ps.setBigDecimal(2, total);
    ps.setString(3, "NEW");
    ps.executeUpdate();
    try (ResultSet keys = ps.getGeneratedKeys()) {
        if (!keys.next()) throw new SQLException("no key generated");
        return keys.getLong(1);
    }
}

// Batched inserts: most drivers return one key per row after executeBatch
try (PreparedStatement ps = con.prepareStatement(sql, new String[] {"id"})) {
    for (NewOrder o : orders) { bind(ps, o); ps.addBatch(); }
    ps.executeBatch();
```


© TechTribers. For personal use only. Do not redistribute.

```
    List<Long> ids = new ArrayList<>();
    try (ResultSet keys = ps.getGeneratedKeys()) {
        while (keys.next()) ids.add(keys.getLong(1));
    }
}
```

> 💡 **Pro tip**
> If you use JPA with `GenerationType.IDENTITY`, Hibernate can't batch inserts, because it needs each generated key immediately. `SEQUENCE` with a pooled optimizer allows batching. The same reasoning applies to hand-written JDBC.

Q814 JD-015 • Intermediate

How do you handle NULL values in JDBC (`setNull`, `wasNull`, `getObject` with a type)?

> ⚡ **Quick answer**
> Primitive getters can't return `null`: `getInt`, `getLong`, `getDouble` and `getBoolean` return `0` / `false` for SQL NULL, and you must call `rs.wasNull()` immediately afterwards to tell the difference. The cleaner option is `rs.getObject("col", Integer.class)` (JDBC 4.1), which returns `null`. For binding, use `ps.setNull(index, Types.X)` or `ps.setObject(index, value, Types.X)`, which handles `null` with an explicit SQL type.

Why the SQL type matters for `setNull`: some databases (and drivers) need to know the parameter type to plan the statement; `setObject(i, null)` without a type fails on several drivers ("could not determine data type of parameter" on PostgreSQL in some contexts).

Remember SQL's three-valued logic when building queries: `WHERE discount_pct = ?` with a bound `null` matches **no rows**, because `x = NULL` is unknown. To match nulls, use `IS NULL`, or `IS NOT DISTINCT FROM ?` (PostgreSQL, SQL standard) / `<=> ?` (MySQL).

Java

```
// Reading
int raw = rs.getInt("discount_pct");          // 0 for NULL AND for a real 0
boolean missing = rs.wasNull();               // must be called right after the getter
Integer discount = rs.getObject("discount_pct", Integer.class);   // null for NULL
String note = rs.getString("note");           // object getters already return null

// Writing
if (discount == null) {
    ps.setNull(3, Types.INTEGER);
} else {
    ps.setInt(3, discount);
}
// shorter, same effect
ps.setObject(3, discount, Types.INTEGER);
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`rs.getInt("discount_pct")` silently turning a missing discount into `0%`, which then flows into pricing logic. Use `getObject(..., Integer.class)` for every nullable numeric column.

Q815 JD-016 • Advanced

How do you map `java.time` types with JDBC and avoid time-zone bugs?

⚡ **Quick answer**

Map by meaning, not by convenience. `DATE` ↔ `LocalDate`, `TIME` ↔ `LocalTime`, `TIMESTAMP` **(without time zone)** ↔ `LocalDateTime`, `TIMESTAMP WITH TIME ZONE` ↔ `OffsetDateTime` (the instant, usually returned in UTC). Use `setObject` / `getObject` (JDBC 4.2) instead of `java.sql.Timestamp` / `Date`, which convert through the **JVM's default time zone** and shift values when servers run in different zones. Store instants (events, audit times) as `timestamptz` /UTC; store calendar values (birthdays, business dates) as `DATE`.

How the classic bug happens: a service running in `Asia/Kolkata` writes a `java.sql.Timestamp` to a `TIMESTAMP` column, another service in `UTC` reads it, and every value is 5.5 hours off, or one day off for dates near midnight. Daylight-saving transitions add duplicated and missing hours.

Defensive habits:

- Run JVMs with `-Duser.timezone=UTC` (and containers with `TZ=UTC`) so the default zone can't vary between hosts.
- Use `timestamptz` (PostgreSQL) or the equivalent for instants; MySQL's `TIMESTAMP` converts using the **session** time zone (set `connectionTimeZone` / `forceConnectionTimeZoneToSession` in Connector/J 8.0.23+), while `DATETIME` stores the literal value.
- Convert to the user's zone only for display.

Driver support for `Instant` and `ZonedDateTime` in `setObject` varies; `OffsetDateTime` is the most portable choice for instants.

```java
// Writing
ps.setObject(1, order.createdAt());                  // OffsetDateTime -> timestamptz
ps.setObject(2, order.deliveryDate());               // LocalDate      -> date
ps.setObject(3, Instant.now().atOffset(ZoneOffset.UTC));   // Instant via OffsetDateTime

// Reading
OffsetDateTime createdAt = rs.getObject("created_at", OffsetDateTime.class);
LocalDate delivery = rs.getObject("delivery_date", LocalDate.class);
Instant asInstant = createdAt.toInstant();

// Legacy API: converts through the JVM default zone (TimeZone.getDefault())
Timestamp ts = rs.getTimestamp("created_at");          // avoid in new code
LocalDateTime local = ts.toLocalDateTime();            // meaning depends on the JVM zone
```


© TechTribers. For personal use only. Do not redistribute.

Map SQL temporal types to java.time by meaning (JDBC 4.2 setObject / getObject)

SQL type	Java type	use it for
`DATE`	LocalDate	birthday, business date
`TIME`	LocalTime	opening hours
`TIMESTAMP`	LocalDateTime	wall-clock value, no zone
`TIMESTAMP WITH TIME ZONE`	OffsetDateTime	an instant: events, audit times
`java.sql.Timestamp`	(legacy)	converts via the JVM default zone ✗

Run JVMs and containers in UTC; convert to the user's zone only for display.

⚠️ **Interview trap**

Mapping `TIMESTAMP WITH TIME ZONE` to `ZonedDateTime` and expecting the original zone back. PostgreSQL stores only the instant; you get it back in UTC (or the session zone), not as `Asia/Kolkata`. If the zone matters, store it in its own column.

🗃️ **Q816** JD-017 • Intermediate

How do you read and write BLOB and CLOB data efficiently?

⚡ **Quick answer**

Stream large values instead of materializing them: bind with `setBinaryStream` / `setCharacterStream` (with a length when you know it) and read with `getBinaryStream` / `getCharacterStream`, copying in chunks. Small values (a few KB) are fine as `byte[]` / `String` via `setBytes` / `getBytes`. Lob locators (`Blob`, `Clob`) are only valid inside the **transaction** that produced them; call `free()` when done. Many teams store large files in object storage (S3) and keep only the key in the database.

Whether streaming really avoids memory use depends on the driver and column type: PostgreSQL `bytea` values are sent inline in the result row, so the driver still holds the whole value; PostgreSQL **large objects** (`oid` + `LargeObjectManager`) and Oracle LOB locators genuinely stream. MySQL `LONGBLOB` values are limited by `max_allowed_packet`.

Production advice:

- Keep large binaries **out of hot tables** (separate table or object storage) so ordinary queries and backups stay small.
- Set limits on upload size before you start streaming into the database.
- Remember that LOB access usually needs an **open transaction** (`autoCommit=false`) on PostgreSQL large objects and Oracle.

```java
// Write a file into a bytea / BLOB column without loading it into memory
try (PreparedStatement ps = con.prepareStatement(
        "INSERT INTO document (id, name, content) VALUES (?, ?, ?)");
```


© TechTribers. For personal use only. Do not redistribute.

```
     InputStream in = Files.newInputStream(path)) {
    ps.setObject(1, UUID.randomUUID());
    ps.setString(2, path.getFileName().toString());
    ps.setBinaryStream(3, in, Files.size(path));
    ps.executeUpdate();
}

// Read it back as a stream
try (PreparedStatement ps = con.prepareStatement(
        "SELECT content FROM document WHERE id = ?")) {
    ps.setObject(1, id);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            try (InputStream in = rs.getBinaryStream(1)) {
                in.transferTo(response.getOutputStream());
            }
        }
    }
}
```

🎯 **Likely follow-up**

How large is too large for a database column? There's no hard rule, but once files are regularly above a few MB, object storage plus a metadata row is usually cheaper, faster to back up and easier to serve via CDN.

© TechTribers. For personal use only. Do not redistribute.

Section 03

ResultSet: Cursors, Streaming and Mapping

How ResultSet cursors and their types work, fetch size and streaming millions of rows, metadata, clean row mapping to records, the getInt-NULL trap, updatable result sets, and query timeouts and cancellation.

Questions 817–823 · 7 questions



Q817 JD-018 • Intermediate

How does a `ResultSet` cursor work, and what are the result set types and concurrency modes?

⚡ Quick answer

A `ResultSet` is a **cursor** positioned **before the first row**; each `next()` moves it forward and returns `false` past the last row. The **type** fixes how you can move and whether you see concurrent changes: `TYPE_FORWARD_ONLY` (default, cheapest), `TYPE_SCROLL_INSENSITIVE` (scroll freely over a snapshot, usually buffered in memory) and `TYPE_SCROLL_SENSITIVE` (scrollable and reflects some changes; rarely supported well). The **concurrency** is `CONCUR_READ_ONLY` (default) or `CONCUR_UPDATABLE` (Q822).

Why forward-only is the default: it lets the driver stream rows and discard them once read. A scroll-insensitive result set usually means the driver keeps **all rows in memory**, and `rs.last()` to count rows reads the entire result. Use `SELECT count()` *or a window function (* `count( ) OVER ()` *)* when you need a total.

`TYPE_SCROLL_SENSITIVE` support is inconsistent: many drivers silently downgrade it (check `rs.getType()` or `SQLWarning`s) and "sensitive" often means only updates you make through the same result set. Don't design around it.

Java

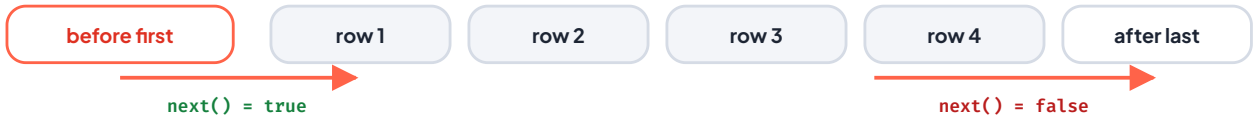
```
// Default: forward-only, read-only - what you want 99% of the time
try (PreparedStatement ps = con.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        handle(rs);
    }
}

// Scrollable: needed only for UI-style navigation or counting on the client
try (PreparedStatement ps = con.prepareStatement(sql,
        ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
     ResultSet rs = ps.executeQuery()) {
    rs.last();
    int rowCount = rs.getRow();          // forces the driver to read everything
    rs.absolute(10);                     // jump to row 10
```


© TechTribers. For personal use only. Do not redistribute.

```
    rs.previous();                    // back to row 9
}
```

ResultSet is a cursor: it starts BEFORE the first row



type	movement	cost / caveat
`TYPE_FORWARD_ONLY`	next() only	streams; default and cheapest
`TYPE_SCROLL_INSENSITIVE`	absolute, previous, last	usually buffers ALL rows
`TYPE_SCROLL_SENSITIVE`	scroll + sees changes	rarely supported; often downgraded

⚠️ **Interview trap**

Paging with `rs.absolute(offset)` on a scrollable result set. The database still produces every row and the driver may transfer them all. Page in SQL with `LIMIT/OFFSET`, or better, keyset pagination (`WHERE id > ? ORDER BY id LIMIT ?`).

Q818 JD-019 • Advanced

What is fetch size, and how do you stream millions of rows without running out of memory?

⚡ **Quick answer**

Fetch size is how many rows the driver requests per round trip while you iterate a `ResultSet`. Defaults differ a lot: **PostgreSQL** reads the **entire result** into memory unless `autoCommit` is **false** and a fetch size is set; **MySQL** also buffers everything by default and streams only with `fetchSize = Integer.MIN_VALUE` (row by row) or `useCursorFetch=true` plus a fetch size; **Oracle** fetches **10 rows** per round trip by default, which is too small for large exports. To stream: forward-only, read-only, a suitable fetch size (often 500–5,000), process and discard each row.

Choosing a fetch size is a trade-off between **round trips** and **memory**: with 1,000,000 rows, a fetch size of 10 means 100,000 round trips (Oracle's default can make large exports painfully slow), while fetching all rows at once means holding them all in the heap. Hundreds to a few thousand rows per fetch is a sensible starting range; measure with your row width.

Side effects of streaming:

- The connection, and on PostgreSQL an **open transaction**, stay busy for the whole export. Long transactions hold back vacuum and keep snapshots alive. Consider exporting from a replica.
- With MySQL row-by-row streaming, you can't run **another query on the same connection** until the stream is fully read or closed.

© TechTribers. For personal use only. Do not redistribute.

- Spring's `JdbcTemplate.setFetchSize` and `queryForStream` apply the same settings; with JPA, use Hibernate's `getResultStream()` plus the fetch-size hint.

Java

```
void exportOrders(Writer out) throws SQLException, IOException {
    try (Connection con = ds.getConnection()) {
        con.setAutoCommit(false);                       // PostgreSQL: required for cursors
        try (PreparedStatement ps = con.prepareStatement(
                "SELECT id, customer_id, total, created_at FROM orders",
                ResultSet.TYPE_FORWARD_ONLY, ResultSet.CONCUR_READ_ONLY)) {
            ps.setFetchSize(1_000);                     // rows per round trip
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    out.write(toCsvLine(rs));           // nothing accumulates in memory
                }
            }
        } finally {
            con.commit();                               // ends the read transaction
        }
    }
}

// MySQL Connector/J streaming alternatives:
//   ps.setFetchSize(Integer.MIN_VALUE);                    // row-by-row streaming
//   or URL useCursorFetch=true with ps.setFetchSize(1000)  // server-side cursor
```

Default fetching per driver — and how to stream a huge result

driver	default	to stream efficiently
PostgreSQL	ALL rows into memory	autoCommit=false + setFetchSize(n)
MySQL	ALL rows into memory	fetchSize=Integer.MIN_VALUE, or useCursorFetch=true
Oracle	10 rows per round trip	setFetchSize(500–5,000): fewer round trips
SQL Server	adaptive buffering	responseBuffering=adaptive (default)

1,000,000 rows: fetch size 10 → 100,000 round trips; fetch everything → whole result in the heap.

Most common batch-job OOM: setFetchSize on PostgreSQL with auto-commit ON (silently ignored).

⚠️ **Interview trap**

Calling `setFetchSize(1000)` on PostgreSQL with auto-commit **on** and assuming it streams. The driver ignores the fetch size and loads every row; this is one of the most common causes of `OutOfMemoryError` in batch jobs.

© TechTribers. For personal use only. Do not redistribute.

Q819 JD-020 • Intermediate

What are `ResultSetMetaData` and `DatabaseMetaData` used for?

⚡ **Quick answer**

`ResultSetMetaData` (from `rs.getMetaData()`) describes the **columns of a result**: count, labels, SQL types, precision, nullability. It powers generic tools: CSV/JSON exporters, generic row mappers, admin consoles. `DatabaseMetaData` (from `con.getMetaData()`) describes the **database and driver**: product and version, supported features and isolation levels, keyword lists, and catalog information (tables, columns, primary keys, indexes). Schema tools such as Flyway, Liquibase and Hibernate's schema validation use it.

Use `getColumnLabel` (the `AS` alias) rather than `getColumnName` (the underlying column) when mapping results; the difference matters for expressions and aliases, and some drivers return the label for both.

Catalog calls (`getTables`, `getColumns`, `getIndexInfo`) can be **slow** on large schemas, and the meaning of catalog and schema differs by database (MySQL uses catalogs for databases; PostgreSQL uses schemas). Call them at startup or in tools, not per request.

Java

```java
// Generic: turn any query into a list of maps (column label -> value)
List<Map<String, Object>> toMaps(ResultSet rs) throws SQLException {
    ResultSetMetaData md = rs.getMetaData();
    int n = md.getColumnCount();
    List<Map<String, Object>> rows = new ArrayList<>();
    while (rs.next()) {
        Map<String, Object> row = new LinkedHashMap<>();
        for (int i = 1; i <= n; i++) {
            row.put(md.getColumnLabel(i), rs.getObject(i));
        }
        rows.add(row);
    }
    return rows;
}

// Database capabilities and catalog
DatabaseMetaData db = con.getMetaData();
boolean serializable = db.supportsTransactionIsolationLevel(
        Connection.TRANSACTION_SERIALIZABLE);
try (ResultSet cols = db.getColumns(null, "public", "orders", "%")) {
    while (cols.next()) {
        System.out.println(cols.getString("COLUMN_NAME") + " "
                + cols.getString("TYPE_NAME"));
    }
}
```

💡 **Pro tip**

`DatabaseMetaData.getSQLKeywords()` and `getIdentifierQuoteString()` help when writing tools that generate SQL for several databases; JDBC 4.3's `enquoteIdentifier` is simpler when you just need to quote one name.

© TechTribers. For personal use only. Do not redistribute.

Q820 JD-021

• Intermediate

How do you map `ResultSet` rows to records cleanly with a RowMapper?

> ⚡ **Quick answer**
>
> Separate **iteration** from **mapping**: a small functional interface `RowMapper<T>` maps the current row to an object (one place per type, reused by every query), and a generic helper owns the loop and resource handling. Records are a natural target: immutable, compact, with a canonical constructor. Read columns by **label**, use `getObject(label, Type.class)` for nullable and `java.time` columns, and map inside the try-with-resources block.

This is exactly the shape of Spring's `JdbcTemplate.query(sql, RowMapper<T>, args...)` and `JdbcClient.sql(...).query(Customer.class)`. Spring also has `DataClassRowMapper`, which maps columns to a record's constructor parameters by name (snake_case to camelCase), saving the hand-written mapper for simple cases.

Tips that keep mappers robust:

- Select explicit columns instead of `SELECT *`, so adding a column to the table doesn't change what you fetch.
- Keep mapping free of side effects; don't run more queries from inside a mapper (that's the N+1 problem, Q840).
- For joins producing parent/child rows, use a `ResultSetExtractor`-style function that sees the whole result and groups rows, rather than a per-row mapper.

Java

```java
@FunctionalInterface
interface RowMapper<T> {
    T map(ResultSet rs) throws SQLException;
}

record Customer(long id, String name, String email, LocalDate since,
                Integer loyaltyTier) {

    static final RowMapper<Customer> MAPPER = rs -> new Customer(
            rs.getLong("id"),
            rs.getString("name"),
            rs.getString("email"),
            rs.getObject("customer_since", LocalDate.class),
            rs.getObject("loyalty_tier", Integer.class));   // nullable column
}

static <T> List<T> query(DataSource ds, String sql, RowMapper<T> mapper,
                         Object... params) throws SQLException {
    try (Connection con = ds.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {
        for (int i = 0; i < params.length; i++) ps.setObject(i + 1, params[i]);
        try (ResultSet rs = ps.executeQuery()) {
            List<T> out = new ArrayList<>();
            while (rs.next()) out.add(mapper.map(rs));
            return out;
        }
    }
```


© TechTribers. For personal use only. Do not redistribute.

```
}

List<Customer> pune = query(ds,
        "SELECT * FROM customer WHERE city = ? ORDER BY name",
        Customer.MAPPER, "Pune");
```

🎯 **Likely follow-up**

How would you map a one-to-many join (order with lines) in one query? Iterate the rows, keep a `LinkedHashMap<Long, OrderBuilder>` keyed by order ID, add each row's line to its order, then build the records at the end.

0️⃣ Q821 JD-022 • Basic

Column index vs column label, and why does `getInt` return `0` for SQL NULL?

⚡ **Quick answer**

Getters accept a **1-based column index** or a **column label**. Index access is marginally faster but breaks silently when the `SELECT` list changes; label access is self-documenting and robust, and it's the right default. Primitive getters return `0`, `0L`, `0.0` or `false` for **SQL NULL**, because a primitive can't hold null; use `wasNull()` or `getObject(label, Integer.class)` to see the difference (Q814).

Label lookup details: labels are matched **case-insensitively**, and if two columns share a label (`SELECT o.id, c.id ...`) the getter returns the **first** one. Always alias duplicates (`o.id AS order_id, c.id AS customer_id`).

The index is 1-based (as are parameter indexes in `PreparedStatement`), a classic off-by-one source for developers used to arrays.

Performance difference between index and label is negligible for typical row counts; drivers cache the label-to-index mapping. Choose labels for maintainability.

Java

```
// Fragile: depends on column order in the SELECT list
String sql = "SELECT id, name, credit_limit FROM customer";
long id = rs.getLong(1);
BigDecimal limit = rs.getBigDecimal(3);     // breaks if someone adds a column at 2

// Robust: by label (use AS aliases for expressions)
String sql2 = """
    SELECT c.id, c.name, c.credit_limit, count(o.id) AS order_count
    FROM customer c LEFT JOIN orders o ON o.customer_id = c.id
    GROUP BY c.id, c.name, c.credit_limit
    """;
long id2 = rs.getLong("id");
long orders = rs.getLong("order_count");

// The NULL trap
int tier = rs.getInt("loyalty_tier");            // 0 when the column is NULL
Integer tierOrNull = rs.getObject("loyalty_tier", Integer.class);
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

A join returning `id` from two tables, read with `rs.getLong("id")`. You silently get whichever `id` comes first in the `SELECT` list, often the wrong entity's ID.

✏️ **Q822** JD-023 • Intermediate

What are updatable result sets, and why are they rarely used?

⚡ **Quick answer**

With `CONCUR_UPDATABLE`, you can change the current row through the cursor (`rs.updateString(...)`, then `rs.updateRow()`), insert rows (`moveToInsertRow`, `insertRow`) or delete them (`deleteRow`). They're rarely used because they only work for simple single-table queries with a primary key, behave differently across drivers (some silently downgrade to read-only), keep cursors and often locks open, and hide the actual SQL. Explicit `UPDATE` statements, or batches of them, are clearer and faster.

Where they still appear: legacy desktop tools, quick admin utilities, and some Oracle code. If you see them in a code review of a service, the set-based `UPDATE` is almost always simpler, atomic and much faster (one statement instead of one per row).

If logic per row is complex, a common middle ground is: select IDs and data, compute in Java, then write back with a **batched** `UPDATE ... WHERE id = ?` (Q833) inside one transaction, possibly with a version column for concurrency safety.

Java

```
try (PreparedStatement ps = con.prepareStatement(
        "SELECT id, status, retries FROM job WHERE status = 'FAILED'",
        ResultSet.TYPE_FORWARD_ONLY, ResultSet.CONCUR_UPDATABLE);
     ResultSet rs = ps.executeQuery()) {
    if (rs.getConcurrency() != ResultSet.CONCUR_UPDATABLE) {
        throw new IllegalStateException("driver downgraded to read-only");
    }
    while (rs.next()) {
        rs.updateString("status", "PENDING");
        rs.updateInt("retries", rs.getInt("retries") + 1);
        rs.updateRow();                      // issues an UPDATE for this row
    }
}

// Usually better: one explicit statement
UPDATE job SET status = 'PENDING', retries = retries + 1 WHERE status = 'FAILED';
```

🎯 **Likely follow-up**

How do you know whether the driver honoured your requested type and concurrency? Check `rs.getType()` and `rs.getConcurrency()` and look at `con.getWarnings()` / `ps.getWarnings()`: drivers report downgrades as `SQLWarning`s.

© TechTribers. For personal use only. Do not redistribute.

Q823 JD-024

• Advanced

How do query timeouts, `setMaxRows` and `Statement.cancel` work?

> ⚡ **Quick answer**
> `setQueryTimeout(seconds)` asks the driver to **cancel** a statement that runs too long and throw `SQLTimeoutException`; it's implemented by a driver timer thread sending a cancel request (PostgreSQL), a `KILL QUERY` (MySQL) or a protocol break (Oracle). `setMaxRows(n)` caps the rows returned; it's a client-side limit unless the driver translates it, so prefer `LIMIT` in SQL. `cancel()` lets another thread abort a running statement. These complement, rather than replace, the **socket timeout** and server-side limits such as `statement_timeout`.

Layering timeouts properly (from inner to outer):

- **Server**: `statement_timeout`, `lock_timeout`, `idle_in_transaction_session_timeout` on PostgreSQL; `max_execution_time` for MySQL `SELECT`s. These protect the database regardless of client behaviour.
- **Statement**: `setQueryTimeout` for per-query budgets from the application.
- **Socket**: driver `socketTimeout` as the last resort against dead connections.
- **Pool**: `connectionTimeout` for how long a caller waits for a free connection.

A cancelled query may still have done work (and held locks) on the server; the transaction usually needs a rollback afterwards. Some drivers can't cancel while waiting on network I/O, which is why the socket timeout still matters.

Java

```
try (PreparedStatement ps = con.prepareStatement(reportSql)) {
    ps.setQueryTimeout(10);                 // seconds; 0 = no limit
    ps.setMaxRows(10_000);                  // safety net, not a paging mechanism
    try (ResultSet rs = ps.executeQuery()) {
        render(rs);
    }
} catch (SQLTimeoutException e) {
    log.warn("report query cancelled after 10s", e);
    throw new ReportTooSlowException(e);
}

// Server-side limits (PostgreSQL), per role or per session
// ALTER ROLE reporting SET statement_timeout = '30s';
// SET LOCAL lock_timeout = '2s';           -- inside a transaction

// Spring: @Transactional(timeout = 10) applies the remaining time as the query timeout
```

> ⚠️ **Interview trap**
> Relying on `setMaxRows(100)` for pagination. The database may still compute and send far more rows; only `LIMIT` / `FETCH FIRST` changes the query plan.

© TechTribers. For personal use only. Do not redistribute.

Section 04

Transactions, Isolation and Locking

The transaction bugs that reach production, savepoints, isolation levels and their anomalies, what each database really does by default, optimistic and pessimistic locking, retrying deadlocks and serialization failures, read-only routing, XA, and how Spring binds connections to transactions.

Questions 824–832 · 9 questions



Q824 JD-025

• Intermediate

What are the most common JDBC transaction bugs, and what does a correct transaction template look like?

⚡ Quick answer

The classic bugs: **forgetting** `rollback()` on failure (the next pool user inherits a half-done transaction or the pool has to clean it up), catching only `SQLException` and missing `RuntimeException`s, **leaving** `autoCommit=false` on a pooled connection, committing after a **swallowed exception**, doing slow remote calls **inside** a transaction, and assuming each statement in auto-commit mode is part of one unit of work. A correct template: borrow, disable auto-commit, do the work, commit, roll back on **any** throwable, restore state, close.

In **auto-commit mode** (the JDBC default) every statement is its own transaction. Three inserts that must succeed together can partially succeed, and every statement pays a commit (on many databases that's a log flush), which also makes bulk loading slow.

Other traps:

- `Connection.close()` with an open transaction: the spec leaves commit-or-rollback **implementation-defined**. HikariCP rolls back uncommitted work when the connection returns to the pool, but Oracle's driver historically **committed** on close. Never rely on close to end a transaction.
- **Long transactions** hold locks, block DDL and (PostgreSQL) prevent vacuum from cleaning up dead rows. Keep transactions short and free of user think-time and remote calls.
- `rethrow` with `throw t` on a `Throwable` compiles here because of Java's precise rethrow: `t` can only be what `work.run` declares or an unchecked type.

Java

```
interface TxWork<T> { T run(Connection con) throws SQLException; }

static <T> T inTransaction(DataSource ds, TxWork<T> work) throws SQLException {
    try (Connection con = ds.getConnection()) {
```


© TechTribers. For personal use only. Do not redistribute.

```
        boolean previous = con.getAutoCommit();
        con.setAutoCommit(false);
        try {
            T result = work.run(con);
            con.commit();
            return result;
        } catch (Throwable t) {                // SQLException, RuntimeException, Error
            try {
                con.rollback();
            } catch (SQLException rollbackFailure) {
                t.addSuppressed(rollbackFailure);
            }
            throw t;
        } finally {
            con.setAutoCommit(previous);       // pools also reset this, but be explicit
        }
    }
}

long orderId = inTransaction(ds, con -> {
    long id = insertOrder(con, order);
    insertLines(con, id, order.lines());
    reserveStock(con, order.lines());
    return id;
});
```

💡 **Pro tip**

Q149 covers the basic commit/rollback API; this template is what a reviewer expects to see in raw JDBC code. In Spring, `TransactionTemplate` and `@Transactional` implement it for you (Q832).

🚩 **Q825** JD-026 • Intermediate

What are savepoints, and when are they useful?

⚡ **Quick answer**

A **savepoint** marks a point inside a transaction that you can roll back to without abandoning the whole transaction: `Savepoint sp = con.setSavepoint(); ... con.rollback(sp);`. It's useful when one optional step may fail but the rest of the work should still commit: importing a batch where a bad record should be skipped, or trying an insert and falling back to an update. On PostgreSQL it's also the **only** way to continue after an error inside a transaction, because any failed statement aborts the transaction until rollback.

Costs: each savepoint is a server-side subtransaction. On PostgreSQL, many subtransactions in one transaction (more than 64 per backend overflow its cache) can hurt performance on busy systems, so a savepoint **per row** for very large imports isn't free; validating rows in Java first, or using `INSERT ... ON CONFLICT DO NOTHING`, is often better.

Savepoints are also how frameworks implement **nested transactions**: Spring's `Propagation.NESTED` with `DataSourceTransactionManager` uses a JDBC savepoint, unlike `REQUIRES_NEW`, which suspends the current

© TechTribers. For personal use only. Do not redistribute.

transaction and starts a separate one on another connection.

```java
con.setAutoCommit(false);
int imported = 0, skipped = 0;
for (CustomerRow row : rows) {
    Savepoint sp = con.setSavepoint();
    try {
        insertCustomer(con, row);
        con.releaseSavepoint(sp);              // optional: frees server resources
        imported++;
    } catch (SQLException e) {
        con.rollback(sp);                      // undo just this row
        skipped++;
        log.warn("skipping row {}: {}", row.lineNo(), e.getMessage());
    }
}
con.commit();                                  // everything that succeeded

// PostgreSQL without the savepoint: after the first failing insert,
// every later statement fails with
// "current transaction is aborted, commands ignored until end of transaction block"
```

🎯 **Likely follow-up**

Does `rollback(sp)` release locks taken after the savepoint? On most databases, yes, locks acquired after the savepoint are released; locks taken earlier are kept until the transaction ends.

Q826 JD-027 • Intermediate

Which isolation levels does JDBC define, and which anomalies does each prevent?

⚡ **Quick answer**

JDBC exposes the four ANSI levels via `con.setTransactionIsolation(...)`: **READ UNCOMMITTED** (dirty reads possible), **READ COMMITTED** (no dirty reads; non-repeatable reads and phantoms possible), **REPEATABLE READ** (no non-repeatable reads; phantoms possible by the ANSI definition), **SERIALIZABLE** (behaves as if transactions ran one after another). Real databases add anomalies the ANSI list misses, notably **lost updates** and **write skew**, and implement the levels differently (Q827).

The anomalies, briefly:

- **Dirty read**: reading another transaction's uncommitted data.
- **Non-repeatable read**: reading the same row twice and getting different values because another transaction committed in between.
- **Phantom read**: re-running a range query and seeing new or vanished rows.
- **Lost update**: two transactions read, modify and write the same row; one write silently overwrites the other.

© TechTribers. For personal use only. Do not redistribute.

- **Write skew**: two transactions read overlapping data, each writes a different row, and together they break an invariant (the "two doctors go off call" example above). Only true serializability, or explicit locking, prevents it.

Set the isolation level **before** the transaction starts; changing it mid-transaction is undefined or rejected. Pooled connections should have their isolation restored afterwards; HikariCP resets it when the connection is returned if you changed it.

Java

```java
try (Connection con = ds.getConnection()) {
    con.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE);  // before any work
    con.setAutoCommit(false);
    try {
        // check invariant, then write: safe only if the DB detects conflicts
        int onCall = countDoctorsOnCall(con, shiftId);
        if (onCall < 2) throw new BusinessException("one doctor must stay on call");
        markOffCall(con, doctorId, shiftId);
        con.commit();
    } catch (SQLException e) {
        con.rollback();
        if ("40001".equals(e.getSQLState())) {
            // serialization failure: retry the whole transaction (Q829)
        }
        throw e;
    }
}

// Constants: TRANSACTION_READ_UNCOMMITTED, TRANSACTION_READ_COMMITTED,
//            TRANSACTION_REPEATABLE_READ, TRANSACTION_SERIALIZABLE
```

ANSI isolation levels vs anomalies (✓ = prevented)

level	dirty read	non-repeatable	phantom	lost update	write skew
READ UNCOMMITTED	✗	✗	✗	✗	✗
READ COMMITTED	✓	✗	✗	✗	✗
REPEATABLE READ	✓	✓	✗ / ✓*	✓*	✗
SERIALIZABLE	✓	✓	✓	✓	✓

* depends on the engine: PostgreSQL REPEATABLE READ is snapshot isolation (no phantoms, aborts lost updates).

Defaults: PostgreSQL, Oracle, SQL Server = READ COMMITTED · MySQL InnoDB = REPEATABLE READ

⚠️ **Interview trap**

Choosing SERIALIZABLE and not writing retry logic. Serializable isolation works by aborting conflicting transactions; without retries it simply turns races into errors.

© TechTribers. For personal use only. Do not redistribute.

Q827 JD-028

Advanced

What isolation level does each major database use by default, and how does MVCC change the picture?

⚡ Quick answer

Defaults: **PostgreSQL, Oracle and SQL Server** use **READ COMMITTED**; **MySQL InnoDB** uses **REPEATABLE READ**. Most use **MVCC** (multi-version concurrency control): readers see a snapshot and don't block writers. So PostgreSQL's REPEATABLE READ is really **snapshot isolation** (no phantoms either, but write skew possible); its SERIALIZABLE is **SSI**, which detects dangerous patterns and aborts with SQLState `40001`. Oracle has no real REPEATABLE READ and its SERIALIZABLE is snapshot isolation. SQL Server is lock-based by default unless `READ_COMMITTED_SNAPSHOT` is on.

Consequences people get wrong:

- **MySQL REPEATABLE READ** uses a consistent snapshot for plain `SELECT`s but **locking reads** (`SELECT ... FOR UPDATE`, `UPDATE`) see the latest committed data and take **gap/next-key locks**, which is why InnoDB blocks phantom inserts in locked ranges and why it deadlocks more on range updates.
- **PostgreSQL READ COMMITTED** re-checks the `WHERE` clause on rows changed by concurrent transactions during an `UPDATE`; under REPEATABLE READ the same situation raises `40001` ("could not serialize access due to concurrent update") instead.
- An **application moving from MySQL to PostgreSQL** effectively moves from REPEATABLE READ to READ COMMITTED; logic that assumed a stable snapshot across statements can change behaviour.

Because readers don't block writers under MVCC, "just raise isolation" costs little for reads but turns write conflicts into **aborts that need retries**.

Java

```
DatabaseMetaData md = con.getMetaData();
int def = md.getDefaultTransactionIsolation();
log.info("{} default isolation = {}", md.getDatabaseProductName(), switch (def) {
    case Connection.TRANSACTION_READ_UNCOMMITTED -> "READ UNCOMMITTED";
    case Connection.TRANSACTION_READ_COMMITTED   -> "READ COMMITTED";
    case Connection.TRANSACTION_REPEATABLE_READ  -> "REPEATABLE READ";
    case Connection.TRANSACTION_SERIALIZABLE     -> "SERIALIZABLE";
    default -> "NONE";
});

-- PostgreSQL
SHOW default_transaction_isolation;                    -- read committed
-- MySQL
SELECT @@transaction_isolation;                        -- REPEATABLE-READ
```

💡 Pro tip

Many teams keep READ COMMITTED and protect specific invariants with explicit locking (`SELECT ... FOR UPDATE`), version columns or database constraints, instead of raising isolation globally.

© TechTribers. For personal use only. Do not redistribute.

Q828 JD-029

• Advanced

Optimistic vs pessimistic locking with plain JDBC (version column, `SELECT ... FOR UPDATE`, `SKIP LOCKED`)

⚡ Quick answer

Optimistic: read the row with its **version**, and update with `WHERE id = ? AND version = ?` while incrementing the version; `0` rows updated means someone else changed it, so you reload or report a conflict. No locks are held between read and write, which suits user edits and low contention.
Pessimistic: lock the row when reading with `SELECT ... FOR UPDATE` (inside a transaction), so concurrent writers **wait**; it suits short, high-contention critical sections like counters and stock. `FOR UPDATE SKIP LOCKED` lets many workers **claim different rows** of a job queue without blocking each other.

Choosing between them:

- **Optimistic** wins when conflicts are rare and the read-to-write gap is long (a user editing a form for minutes). JPA's `@Version` is exactly this pattern.
- **Pessimistic** wins when conflicts are frequent and the critical section is short; optimistic locking would just keep retrying. Keep these transactions very short, and lock rows in a **consistent order** (for example by ID) to avoid deadlocks.
- **Atomic updates** often remove the need for either: `UPDATE inventory SET stock = stock - ? WHERE sku = ? AND stock >= ?` checks and decrements in one statement; `rows == 0` means insufficient stock.

`NOWAIT` (fail immediately if locked) and `lock_timeout` are useful when waiting isn't acceptable. `SKIP LOCKED` is supported on PostgreSQL 9.5+, MySQL 8.0+ and Oracle.

Java

```java
// Optimistic: no lock, detect the conflict at write time
int updated;
try (PreparedStatement ps = con.prepareStatement("""
        UPDATE product SET price = ?, version = version + 1
        WHERE id = ? AND version = ?""")) {
    ps.setBigDecimal(1, newPrice);
    ps.setLong(2, p.id());
    ps.setInt(3, p.version());                    // the version we originally read
    updated = ps.executeUpdate();
}
if (updated == 0) throw new ConcurrentModificationException("product " + p.id());

// Pessimistic: lock the row for the rest of the transaction
try (PreparedStatement ps = con.prepareStatement(
        "SELECT stock FROM inventory WHERE sku = ? FOR UPDATE")) {
    ps.setString(1, sku);
    // ... read stock, check, then UPDATE inventory in the same transaction
}

// Job queue: each worker claims up to 10 jobs nobody else has locked
String claim = """
    SELECT id, payload FROM job
    WHERE status = 'READY'
    ORDER BY id
```


© TechTribers. For personal use only. Do not redistribute.

```
    LIMIT 10
    FOR UPDATE SKIP LOCKED""";
```

Optimistic vs pessimistic locking

Optimistic (version column)

```
1  SELECT ..., version  (v = 7)
2  user edits, no lock held
3  UPDATE ... SET version = 8
   WHERE id = ? AND version = 7
4  0 rows → conflict: reload
```

Pessimistic (row lock)

```
1  BEGIN
2  SELECT ... FOR UPDATE  (lock)
3  others WAIT on this row
4  UPDATE ...
5  COMMIT  (lock released)
```

Rare conflicts, long think time → optimistic. Hot rows, short critical section → pessimistic.

Job queues: FOR UPDATE SKIP LOCKED lets each worker claim different rows without blocking.

⚠️ **Interview trap**

`SELECT ... FOR UPDATE` in auto-commit mode. The lock is released as soon as that single statement's transaction ends, so it protects nothing.

🔁 **Q829** JD-030 • Advanced

How do you handle deadlocks and serialization failures with retries?

⚡ **Quick answer**

Both are **expected** outcomes of concurrency, not bugs to log and forget. The database aborts one transaction (deadlock victim, or a serialization failure under REPEATABLE READ/SERIALIZABLE) and reports it with a **SQLState** (`40001` serialization failure; PostgreSQL uses `40P01` for deadlocks; MySQL reports deadlocks as error 1213 with `40001`). The fix is to **retry the whole transaction** from the beginning, with a bounded number of attempts and jittered backoff, and to make the transaction **idempotent** or safe to re-run.

Reducing deadlocks in the first place:

- Access rows and tables in a **consistent order** in every code path (sort IDs before locking or updating several rows).
- Keep transactions **short**, and don't hold locks across user interaction or remote calls.
- Index foreign keys and the columns used in `UPDATE ... WHERE`; on InnoDB, missing indexes turn row locks into large range locks.
- Read the database's deadlock report (`SHOW ENGINE INNODB STATUS`, PostgreSQL's log with `log_lock_waits`) to see the two statements involved.

Retry only what's safe: the whole transaction, never a single statement inside it (the transaction is already rolled back), and never non-transactional side effects such as sending emails, which belong after commit or in an

© TechTribers. For personal use only. Do not redistribute.

outbox. Spring Retry / Resilience4j or Spring 7's built-in `@Retryable` can wrap the transactional method, provided the retry sits **outside** the transaction boundary.

```java
static final Set<String> RETRYABLE = Set.of("40001", "40P01");

static <T> T withRetry(DataSource ds, TxWork<T> work) throws SQLException {
    int maxAttempts = 4;
    for (int attempt = 1; ; attempt++) {
        try {
            return inTransaction(ds, work);          // a NEW transaction each attempt
        } catch (SQLException e) {
            boolean retryable = RETRYABLE.contains(e.getSQLState())
                    || e instanceof SQLTransactionRollbackException;
            if (!retryable || attempt == maxAttempts) throw e;
            long backoff = (long) (Math.pow(2, attempt) * 10
                    + ThreadLocalRandom.current().nextInt(20));
            log.info("retrying after {} (attempt {})", e.getSQLState(), attempt);
            sleepQuietly(backoff);
        }
    }
}
```

⚠️ **Interview trap**

Retrying inside a `@Transactional` method. The retry runs in the same, already-doomed transaction; the annotation order must put the retry around the transaction.

📖 **Q830** JD-031 • Intermediate

What does `setReadOnly(true)` do, and how do you route reads to replicas?

⚡ **Quick answer**

`con.setReadOnly(true)` is a **hint** to the driver that the transaction won't write. What it does depends on the driver: pgjdbc sends `SET SESSION CHARACTERISTICS AS TRANSACTION READ ONLY`-style settings (writes then fail), MySQL Connector/J can route to a replica when using a replication-aware URL, and some drivers ignore it. **Read/write routing** is usually done above JDBC: an `AbstractRoutingDataSource` (Spring) or separate pools for primary and replica, chosen per transaction, with awareness of **replication lag**.

Why `LazyConnectionDataSourceProxy`: the transaction manager normally fetches the connection when the transaction begins, before the read-only flag is visible to the router. The proxy delays fetching until the first statement runs.

Replication lag is the design problem, not the routing: a user who just placed an order and is redirected to "my orders" may not see it if that read goes to a replica a second behind. Common answers are **read-your-writes** routing (send a user's reads to the primary for a short time after they write), session consistency tokens, or simply keeping user-facing reads on the primary and sending only reports and analytics to replicas.

© TechTribers. For personal use only. Do not redistribute.

`readOnly = true` also lets Hibernate skip dirty checking and flushing, which is a separate performance benefit.

```java
// Spring: route by the read-only flag of the current transaction
class ReplicaRoutingDataSource extends AbstractRoutingDataSource {
    @Override
    protected Object determineCurrentLookupKey() {
        return TransactionSynchronizationManager.isCurrentTransactionReadOnly()
                ? "replica" : "primary";
    }
}

// Wrap it so the connection is fetched only after the tx attributes are known
@Bean
DataSource dataSource(DataSource primary, DataSource replica) {
    var routing = new ReplicaRoutingDataSource();
    routing.setTargetDataSources(Map.of("primary", primary, "replica", replica));
    routing.setDefaultTargetDataSource(primary);
    routing.afterPropertiesSet();
    return new LazyConnectionDataSourceProxy(routing);
}

@Transactional(readOnly = true)
public List<OrderSummary> dashboard(long customerId) { ... }   // goes to the replica
```

🎯 **Likely follow-up**

What happens if code in a read-only transaction writes? On PostgreSQL with pgjdbc honouring read-only, the statement fails with `cannot execute INSERT in a read-only transaction`; on a replica it fails because the server is read-only. Both are good: they surface mis-annotated methods early.

Q831 JD-032 • Advanced

What are XA distributed transactions, and why do microservices usually avoid them?

⚡ **Quick answer**

XA is the X/Open standard for **two-phase commit (2PC)** across several resources (two databases, or a database and a JMS broker). In JDBC, an `XADataSource` provides `XAConnection`s whose `XAResource` is enlisted by a **transaction manager** (Narayana, Atomikos, a Jakarta EE server) that runs *prepare* on all participants and then *commit*. Microservices avoid it because it couples services' availability, holds locks across the network while waiting, needs a coordinator with durable logs and recovery, and isn't supported by many modern brokers and cloud databases. They use **local transactions plus the outbox pattern and sagas** instead.

What 2PC gives you and what it costs:

- **Gives**: atomic commit across resources, under the assumption that the coordinator and participants eventually recover.

© TechTribers. For personal use only. Do not redistribute.

- **Costs**: a blocking protocol. If the coordinator dies after *prepare*, participants keep **in-doubt** transactions holding locks until recovery; operations teams then have to resolve them manually. Latency rises by extra round trips and forced log writes.

The outbox pattern turns "write to the database and publish an event" into one local transaction plus reliable, **at-least-once** delivery. Combined with idempotent consumers and sagas (compensating actions for multi-service workflows), it provides eventual consistency without a distributed lock manager.

XA is still reasonable inside a single application talking to two transactional resources, for example a database and IBM MQ in a bank's integration layer.

Java

```
// Outbox: one LOCAL transaction writes the business change and the event
inTransaction(ds, con -> {
    long orderId = insertOrder(con, order);
    try (PreparedStatement ps = con.prepareStatement("""
            INSERT INTO outbox (id, aggregate_id, type, payload, created_at)
            VALUES (?, ?, ?, ?::jsonb, now())""")) {
        ps.setObject(1, UUID.randomUUID());
        ps.setLong(2, orderId);
        ps.setString(3, "OrderPlaced");
        ps.setString(4, json(order));
        ps.executeUpdate();
    }
    return orderId;
});
// A relay (polling or CDC with Debezium) publishes outbox rows to Kafka
// at least once; consumers must be idempotent.
```

⚠️ **Interview trap**

Assuming Kafka transactions plus a database transaction add up to an atomic commit. They're two separate transactions; without XA (which Kafka doesn't support) you need the outbox or an idempotent consumer pattern.

Q832 JD-033 • Advanced

How does Spring's `@Transactional` map onto JDBC connections?

⚡ **Quick answer**

`@Transactional` is applied by a **proxy** around the bean (see Q774 for the self-invocation trap). On entry, `DataSourceTransactionManager` (or `JpaTransactionManager`) borrows **one** `Connection` from the `DataSource`, sets `autoCommit=false`, applies isolation, read-only flag and timeout, and **binds** the connection to the current thread through `TransactionSynchronizationManager`. `JdbcTemplate`, `JdbcClient` and Hibernate called inside the method fetch the connection with `DataSourceUtils.getConnection`, so they all **share** it. On exit the manager commits, or rolls back on `RuntimeException` / `Error`, then restores the connection's settings and returns it to the pool.

Details that decide behaviour:

© TechTribers. For personal use only. Do not redistribute.

- **Checked exceptions don't trigger rollback by default**; add `rollbackFor = Exception.class` if your checked exceptions mean failure.
- **Propagation**: `REQUIRED` joins the bound connection; `REQUIRES_NEW` suspends it and borrows a **second** connection (watch for pool exhaustion when many threads do this at once); `NESTED` uses a savepoint on the same connection.
- **Thread binding** means work handed to another thread (an `@Async` method, a `CompletableFuture`, a parallel stream) runs **outside** the transaction, on a different connection.
- Connections bound by `DataSourceUtils` must be released with `DataSourceUtils.releaseConnection`, not `close()`, if you obtain them manually.

With `spring.jpa.open-in-view=true` (Spring Boot's default, with a warning) a connection can stay attached to the request for its entire duration once acquired, which reduces effective pool capacity under load.

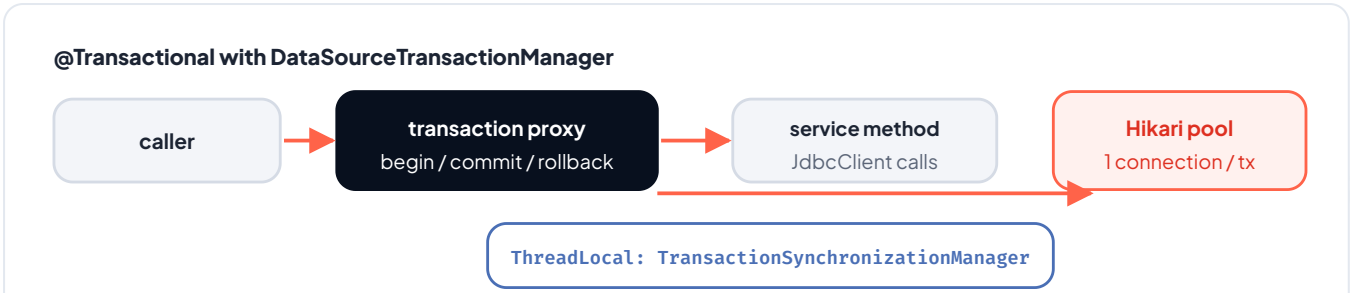
Java

```java
@Service
class CheckoutService {
    private final JdbcClient jdbc;                      // Spring 6.1+
    CheckoutService(JdbcClient jdbc) { this.jdbc = jdbc; }

    @Transactional(isolation = Isolation.READ_COMMITTED, timeout = 5)
    public long placeOrder(NewOrder o) {
        KeyHolder keys = new GeneratedKeyHolder();
        jdbc.sql("INSERT INTO orders (customer_id, total) VALUES (?, ?)")
                .params(o.customerId(), o.total())
                .update(keys, "id");                    // returns the row count
        long id = keys.getKey().longValue();
        jdbc.sql("UPDATE inventory SET stock = stock - ? WHERE sku = ?")
                .params(o.quantity(), o.sku())
                .update();                              // same connection, same tx
        return id;
    }
}

// Equivalent raw-JDBC shape of what the proxy does:
// con = ds.getConnection(); con.setAutoCommit(false); bind(con)
// try { result = placeOrder(...); con.commit(); }
// catch (RuntimeException | Error e) { con.rollback(); throw e; }
// finally { unbind(); reset(con); con.close(); }
```


© TechTribers. For personal use only. Do not redistribute.



1. borrow connection, setAutoCommit(false), apply isolation / readOnly / timeout
2. bind it to the current thread → every JdbcTemplate / JdbcClient / Hibernate call shares it
3. commit, or rollback on RuntimeException / Error (checked exceptions commit by default!)
4. restore settings, unbind, return the connection to the pool

💡 **Pro tip**

Turn on `logging.level.org.springframework.jdbc.datasource.DataSourceTransactionManager=DEBUG` to see `Acquired Connection`, `Switching JDBC Connection to manual commit` and `Committing JDBC transaction` lines; it settles most "is this running in a transaction?" debates.

Section 05

Batching, Connection Pooling and Performance



Batch inserts and the driver flags that make them fast, partial batch failures, how HikariCP works, sizing a pool with real reasoning, the settings that matter, finding connection leaks, N+1 at the JDBC level, and what virtual threads change.

Questions 833–841 · 9 questions

📦 **Q833** JD-034 • Intermediate

How does JDBC batch processing work, and how large should a batch be?

⚡ **Quick answer**

With batching you bind parameters repeatedly and call `addBatch()` for each row, then `executeBatch()` sends them together, returning an `int[]` of update counts. It cuts **round trips** (and, with rewriting, statement executions) dramatically compared with one `executeUpdate()` per row. Use it inside a **transaction** with auto-commit off, flush every **few hundred to a few thousand rows** to bound memory, and commit per batch or per chunk depending on how much you want to redo after a failure.

© TechTribers. For personal use only. Do not redistribute.

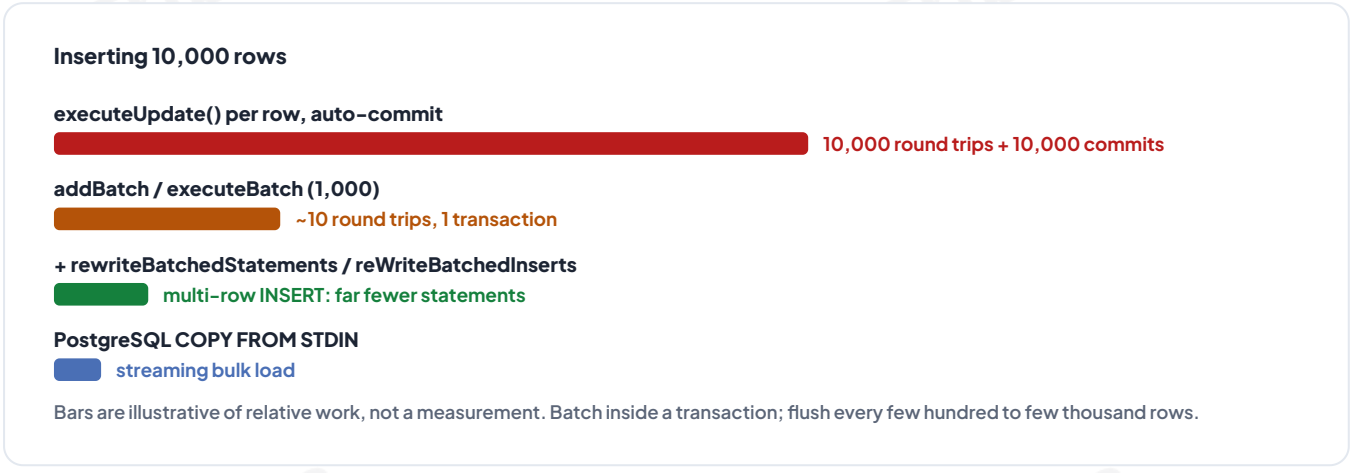
Why it's faster: 10,000 single-row statements cost 10,000 network round trips and, in auto-commit mode, 10,000 commits (each a log flush). A batch sends many rows per round trip, and one transaction means one flush. Gains of 10–100× are common; the exact figure depends on latency, the driver and whether it rewrites the batch (next question).

Choosing the size: beyond a few thousand rows per batch, the gain flattens while memory, lock duration and the cost of a failed batch keep growing. Driver limits also apply (PostgreSQL's 65,535 bind parameters per statement matters when the driver rewrites rows into a multi-row `INSERT`).

Spring: `JdbcTemplate.batchUpdate(sql, list, batchSize, setter)` chunks for you; Hibernate needs `hibernate.jdbc.batch_size` and `order_inserts/order_updates` to batch at all.

Java

```java
void importPrices(List<PriceRow> rows) throws SQLException {
    String sql = "UPDATE product SET price = ?, updated_at = now() WHERE sku = ?";
    try (Connection con = ds.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {
        con.setAutoCommit(false);
        int pending = 0;
        for (PriceRow r : rows) {
            ps.setBigDecimal(1, r.price());
            ps.setString(2, r.sku());
            ps.addBatch();
            if (++pending == 1_000) {           // flush in chunks
                ps.executeBatch();
                con.commit();                   // optional: commit per chunk
                pending = 0;
            }
        }
        if (pending > 0) ps.executeBatch();
        con.commit();
    }
}
```



© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Batching with auto-commit **on**. Many drivers then commit after every statement in the batch, and a failure halfway leaves half the rows committed with no clean way to resume.

⚡ **Q834** JD-035 • Advanced

What do `rewriteBatchedStatements` (MySQL) and `reWriteBatchedInserts` (PostgreSQL) do?

⚡ **Quick answer**
By default some drivers still send a JDBC batch as **many individual statements**, just pipelined. These flags make the driver **rewrite** a batch of single-row `INSERT` s into **multi-row** statements (`INSERT ... VALUES (...), (...), (...)`), so the server parses and executes far fewer statements. MySQL Connector/J: `rewriteBatchedStatements=true` (also rewrites `UPDATE` / `DELETE` batches into multi-statement packets). pgjdbc: `reWriteBatchedInserts=true` (inserts only). They're **off by default**, and for MySQL bulk inserts turning them on often makes the largest single difference.

Side effects to know:

- **Update counts**: with rewriting, drivers may return `Statement.SUCCESS_NO_INFO` (`-2`) for each row instead of real counts, because one multi-row statement produced them. Code that checks every count must accept `-2` .
- **Generated keys** still work for rewritten inserts on current drivers, but verify with your version if you rely on them.
- **Statement size**: MySQL rewriting is bounded by `max_allowed_packet` ; the driver splits automatically.
- PostgreSQL's fastest path for really large loads is `COPY ... FROM STDIN` through pgjdbc's `CopyManager` , which is faster again than rewritten inserts.

These are classic "one-line fix" interview answers for "our batch import takes an hour".

Terminal

```
# MySQL Connector/J
jdbc:mysql://db:3306/shop?rewriteBatchedStatements=true

# PostgreSQL pgjdbc
jdbc:postgresql://db:5432/shop?reWriteBatchedInserts=true

# Spring Boot with Hikari: pass driver properties
spring.datasource.hikari.data-source-properties.rewriteBatchedStatements=true
spring.datasource.hikari.data-source-properties.reWriteBatchedInserts=true
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> When comparing approaches, measure in your environment: with a database a few milliseconds away, round-trip savings dominate; with a local database, server-side parsing and index maintenance dominate.

💔 Q835 JD-036 • Advanced

How do you handle `BatchUpdateException` and partial batch failures?

> ⚡ **Quick answer**
> If a statement in a batch fails, `executeBatch()` throws `BatchUpdateException`. Its `getUpdateCounts()` (or `getLargeUpdateCounts()`) tells you what happened, but drivers differ: some **stop at the first failure** (the array is shorter than the batch), others **continue** and mark failed entries with `Statement.EXECUTE_FAILED` (`-3`). Inside a transaction the safest approach is: roll back, then either fix the data and retry or fall back to row-by-row (or smaller batches) to isolate the bad rows.

Things to check in your driver's behaviour:

- **PostgreSQL** reports the failing statement and attaches the real error via `getNextException()`; the transaction is aborted, so everything must be rolled back.
- **MySQL** with `continueBatchOnError=true` (the Connector/J default) keeps executing the rest and marks failures.
- With rewritten batches (previous question), one bad row fails a whole multi-row statement, so positions become coarse.

Designing around failures is usually better than diagnosing them afterwards: validate rows before inserting, use `INSERT ... ON CONFLICT DO NOTHING/UPDATE` (PostgreSQL) or `INSERT ... ON DUPLICATE KEY UPDATE` (MySQL) for idempotent loads, and load into a staging table first when input quality is poor.

Java

```java
try {
    ps.executeBatch();
    con.commit();
} catch (BatchUpdateException e) {
    con.rollback();
    int[] counts = e.getUpdateCounts();
    if (counts.length < rows.size()) {
        // driver stopped at the first failure: row index = counts.length
        log.error("batch failed at row {}: {}", counts.length, e.getMessage());
    } else {
        for (int i = 0; i < counts.length; i++) {
            if (counts[i] == Statement.EXECUTE_FAILED) {
                log.error("row {} failed", i);
            }
        }
    }
    SQLException cause = e.getNextException();         // the detailed server error
    if (cause != null) log.error("cause: {}", cause.getMessage());
```


© TechTribers. For personal use only. Do not redistribute.

```
    throw e;
}
```

⚠️ **Interview trap**

Logging only `e.getMessage()` of a `BatchUpdateException` from PostgreSQL. It says something like "Batch entry 17 ... was aborted" and tells you to call `getNextException` for the real cause, such as the unique-constraint violation.

Q836 JD-037 • Advanced

What is connection pooling, and how does HikariCP work internally?

⚡ **Quick answer**

A pool keeps a set of **open physical connections** and lends them out: `getConnection()` returns a **proxy** wrapping a pooled connection, and `close()` on the proxy returns it instead of closing the socket. HikariCP keeps connections in a `ConcurrentBag`: a thread first checks its own **thread-local list** of recently used connections, then a shared list, then **waits** on a hand-off queue up to `connectionTimeout`. Background tasks maintain `minimumIdle`, retire connections older than `maxLifetime`, and run keep-alive checks. On return, Hikari **resets** changed state (auto-commit, isolation, read-only, schema) and rolls back open transactions.

Why HikariCP is the default in Spring Boot: small code base, few locks on the hot path (the thread-local fast path means a thread usually gets back the connection it just returned, which is also good for the driver's statement cache), aggressive but correct state resetting, and fast failure (`SQLTransientConnectionException` after `connectionTimeout` instead of hanging).

It also validates connections cheaply: JDBC 4's `Connection.isValid()` rather than a test query, and only for connections that have been idle for a while (by default 500 ms), so the hot path doesn't pay a round trip.

The pool cannot make the database faster. If all connections are busy, extra requests **queue** in the pool, and the right reaction is usually to make the queries or transactions shorter, not to raise the pool size (next question).

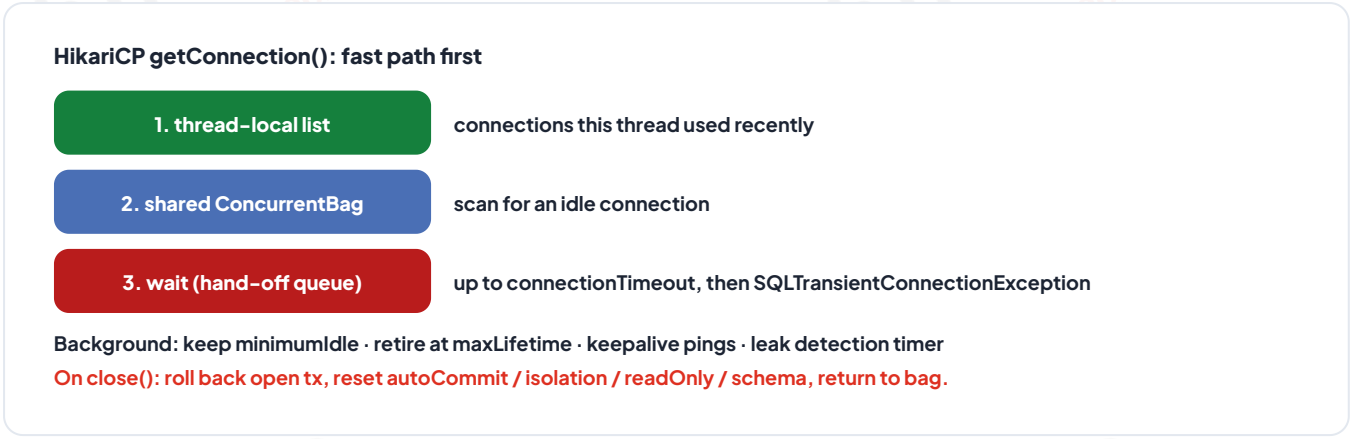
Java

```
HikariConfig cfg = new HikariConfig();
cfg.setJdbcUrl(url);
cfg.setUsername(user);
cfg.setPassword(password);
cfg.setPoolName("orders-db");
cfg.setMaximumPoolSize(10);
cfg.setConnectionTimeout(2_000);          // ms a caller may wait for a connection
cfg.setMaxLifetime(25 * 60_000);          // below the DB's/proxy's idle cut-off
cfg.setKeepaliveTime(2 * 60_000);
cfg.setLeakDetectionThreshold(30_000);    // log holders of a connection > 30 s
cfg.setMetricRegistry(meterRegistry);     // Micrometer: active, idle, pending, wait time

try (HikariDataSource ds = new HikariDataSource(cfg)) {
    HikariPoolMXBean pool = ds.getHikariPoolMXBean();
    log.info("active={} idle={} waiting={}", pool.getActiveConnections(),
```


© TechTribers. For personal use only. Do not redistribute.

```
            pool.getIdleConnections(), pool.getThreadsAwaitingConnection());
}
```



💡 **Pro tip**

Export `hikaricp.connections.pending` and `hikaricp.connections.acquire` (wait time) to your dashboards. Rising pending counts are the earliest sign of database saturation or a connection leak.

📐 **Q837** JD-038 • Advanced

How do you size a database connection pool?

⚡ **Quick answer**

Size it to what the **database** can execute in parallel, not to the number of request threads. HikariCP's well-known starting point is **connections ≈ (CPU cores × 2) + effective spindles** on the database server, typically just **10–20** for a whole database. Check it with **Little's law**: required connections ≈ throughput × time each request holds a connection (for example 500 requests/s × 10 ms = 5 busy connections on average). Then divide the database's budget across **all** application instances, and leave headroom for migrations, admin tools and replicas.

Why bigger isn't better: a database server with 8 cores can only run about 8 queries truly in parallel. Beyond a modest multiple of that, extra connections add context switching, lock contention and memory, and latency for everyone rises. Several published tests (including the PostgreSQL wiki and HikariCP's "About Pool Sizing" page) show throughput dropping as pool size grows past that point.

Practical checks:

- Watch **pool wait time** and **database CPU**. Waiting with low database CPU means connections are being held while doing non-database work (long transactions, remote calls inside transactions, leaks): fix the code. Waiting with high database CPU means the database is saturated: optimise queries or scale the database.
- Remember **rolling deployments** briefly double the instances.
- Pools for different workloads (OLTP vs reporting) are better kept separate so a slow report can't starve checkout.
- `minimumIdle = maximumPoolSize` (Hikari's default) gives a fixed-size pool, which HikariCP recommends for predictable latency.

© TechTribers. For personal use only. Do not redistribute.

```java
// Little's law back-of-the-envelope
double requestsPerSecond = 800;          // peak, per instance
double holdSecondsPerRequest = 0.012;    // time a request holds a connection (p95)
double busy = requestsPerSecond * holdSecondsPerRequest;     // = 9.6 on average
int poolSize = (int) Math.ceil(busy * 1.5);                  // headroom for bursts -> 15

// Budget across the fleet: database max_connections = 200
//   reserved for admin/migrations/monitoring: 20
//   instances (incl. rolling-deploy overlap): 12
//   per-instance maximum: (200 - 20) / 12 = 15
```

Sizing a pool: start from the database, not from request threads

connections ≈ cores × 2	Little's law	fleet budget
+ effective spindles (DB server)	throughput × hold time	(max_conn − reserve) ÷ instances

```
800 req/s × 12 ms held = 9.6 busy connections → pool of ~15 per instance
(200 max_connections - 20 reserved) ÷ 12 instances incl. rolling-deploy overlap = 15
```

Pool waits + LOW DB CPU → connections held too long (tx with remote calls, leaks): fix code.
Pool waits + HIGH DB CPU → database saturated: optimise queries or scale the DB, not the pool.

⚠️ **Interview trap**

Setting the pool to 200 "because we have 200 Tomcat threads". With 10 instances that's 2,000 connections against a database that can usefully run a few dozen queries at once.

Q838 JD-039 • Intermediate

Which HikariCP settings matter in production?

⚡ **Quick answer**

`maximumPoolSize` (default 10) caps connections; `minimumIdle` (defaults to the maximum, giving a fixed pool); `connectionTimeout` (default 30 s) is how long a caller waits for a connection, usually worth lowering to 1–5 s so overload fails fast; `maxLifetime` (default 30 min) retires connections and must be **shorter** than any database, proxy or load-balancer idle cut-off; `idleTimeout` (10 min) applies only when `minimumIdle` is below the maximum; `keepaliveTime` pings idle connections to stop firewalls dropping them; `leakDetectionThreshold` (off by default) logs long-held connections.

Settings that cause incidents when wrong:

- `maxLifetime` **longer than an upstream timeout**: an AWS NLB, Azure load balancer or MySQL `wait_timeout` silently drops idle TCP connections; the pool then hands out dead connections ("Connection reset", "Communications link failure"). Keep `maxLifetime` a few minutes below the shortest such timeout.

© TechTribers. For personal use only. Do not redistribute.

- `connectionTimeout` **of 30 s** behind an HTTP timeout of 10 s: requests have already failed for the user while threads still wait for connections, which amplifies overload.
- `autoCommit` : Hikari's default is `true` . Spring's transaction manager switches it per transaction, but setting `auto-commit=false` together with Hibernate's `provider_disables_autocommit=true` saves a round trip per transaction.

Add the database-side counterpart: PostgreSQL `idle_in_transaction_session_timeout` kills sessions that sit in an open transaction, which protects the database from leaked transactions.

Terminal

```
# Spring Boot application.properties
spring.datasource.hikari.pool-name=orders-db
spring.datasource.hikari.maximum-pool-size=15
spring.datasource.hikari.minimum-idle=15
spring.datasource.hikari.connection-timeout=3000
spring.datasource.hikari.max-lifetime=1500000      # 25 min < proxy idle cut-off
spring.datasource.hikari.keepalive-time=120000
spring.datasource.hikari.leak-detection-threshold=60000
spring.datasource.hikari.validation-timeout=2000
spring.datasource.hikari.data-source-properties.ApplicationName=orders-service
spring.datasource.hikari.data-source-properties.socketTimeout=30
```

🎯 **Likely follow-up**

How do you change pool size without a restart? HikariCP's `HikariConfigMXBean` allows changing `maximumPoolSize` and a few timeouts at run time via JMX; most teams still prefer configuration plus a rolling restart.

Q839 JD-040 • Intermediate

How do you detect and fix connection leaks?

⚡ **Quick answer**

A **leak** is a borrowed connection that's never returned (missing `close()` , a `ResultSet` returned from a method, an exception path without try-with-resources, a stream not closed). Symptoms: active connections climb to the maximum and stay there, callers time out with `SQLTransientConnectionException: Connection is not available, request timed out` , while the database shows **idle** sessions. Detect with HikariCP's `leakDetectionThreshold` , which logs the **stack trace of the borrower**, and with pool metrics; fix by putting every borrow into try-with-resources or a framework template.

Leak detection is a **timer**, so it also flags slow but legitimate work (a 90-second export); set the threshold above your longest normal hold time, and treat each report as either a leak or a transaction that's too long.

Other places to look:

- Database side: `pg_stat_activity` with `state = 'idle'` or `'idle in transaction'` and old `state_change` , filtered by `application_name` .
- Thread dumps: threads blocked in `HikariPool.getConnection` show the symptom; the leak is elsewhere.

© TechTribers. For personal use only. Do not redistribute.

- Code review: any `getConnection()` outside try-with-resources, any `ResultSet` or JDBC `Stream` returned to a caller.

Pools can't safely reclaim a leaked connection automatically (the borrower might still use it), which is why HikariCP only reports leaks.

```java
// LEAK: the connection escapes when an exception happens before close()
Connection con = ds.getConnection();
PreparedStatement ps = con.prepareStatement(sql);
ResultSet rs = ps.executeQuery();      // throws -> con is never closed
process(rs);
con.close();

// LEAK: a lazily-consumed stream that callers forget to close
Stream<Order> orders = jdbcTemplate.queryForStream(sql, mapper);   // holds a connection

// FIX: scope every resource; close the stream explicitly
try (Stream<Order> s = jdbcTemplate.queryForStream(sql, mapper)) {
    s.forEach(this::export);
}

// Hikari log with leakDetectionThreshold=60000:
// WARN ProxyLeakTask - Connection leak detection triggered for
//   org.postgresql.jdbc.PgConnection@1b2c3d on thread http-nio-8080-exec-7,
//   stack trace follows
//   java.lang.Exception: Apparent connection leak detected
//     at com.acme.report.ReportDao.load(ReportDao.java:42)
```

⚠️ **Interview trap**

Raising `maximumPoolSize` to "fix" a leak. It only delays the outage, now with more idle database sessions held open.

Q840 JD-041

• Intermediate

How do you spot and fix the N+1 query problem at the JDBC level?

⚡ **Quick answer**

N+1 is one query for a list followed by **one extra query per row** to load related data: 1 query for 100 orders plus 100 queries for their lines. Each query is fast, but the total is dominated by round trips. Spot it by **counting statements** per request (driver or datasource-proxy logging, APM traces, Hibernate statistics). Fix with a **join** (one query, group rows in Java), an `IN` / `ANY` **batch query** for the children (two queries in total), or by fetching aggregated data in SQL.

Join vs second query: a join returns everything in one round trip but **repeats parent columns** for every child row and multiplies rows when you join several collections (a cartesian explosion). Two queries (parents, then children by `IN` / `ANY`) avoid that and are usually the better default for one-to-many with several children.

© TechTribers. For personal use only. Do not redistribute.

N+1 also hides behind abstractions: a repository call inside a loop, a lazy JPA association touched in a template, or a GraphQL resolver per field. Tooling that **fails tests** when a request exceeds a statement budget (for example datasource-proxy's query-count assertions, or Hypersistence Optimizer for JPA) catches regressions early.

Java

```java
// N+1: one query for orders, then one per order for its lines
List<Order> orders = query("SELECT id, total FROM orders WHERE customer_id = ?", ...);
for (Order o : orders) {
    o.setLines(query("SELECT * FROM order_line WHERE order_id = ?", o.id()));   // N times
}

// Fix: two queries regardless of N
List<Order> orders2 = query("SELECT id, total FROM orders WHERE customer_id = ?", ...);
Long[] ids = orders2.stream().map(Order::id).toArray(Long[]::new);
Map<Long, List<OrderLine>> linesByOrder = new HashMap<>();
try (PreparedStatement ps = con.prepareStatement(
        "SELECT order_id, sku, qty, price FROM order_line WHERE order_id = ANY(?)")) {
    ps.setArray(1, con.createArrayOf("bigint", ids));
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            linesByOrder.computeIfAbsent(rs.getLong("order_id"), k -> new ArrayList<>())
                        .add(OrderLine.MAPPER.map(rs));
        }
    }
}
orders2.forEach(o -> o.setLines(linesByOrder.getOrDefault(o.id(), List.of())));
```

> 💡 **Pro tip**
> Log statement counts per request in development (datasource-proxy or p6spy). A single page producing 200 statements is visible immediately, long before it's slow in production.

🧵 **Q841** JD-042 • Advanced

How do virtual threads interact with JDBC drivers and connection pools?

> ⚡ **Quick answer**
> Virtual threads let you run a blocking JDBC call per request without tying up a platform thread while waiting for the database. Two caveats matter. First, the **pool is still the limit**: 10,000 virtual threads still share, say, 20 connections, so the pool becomes a queue (that's correct back-pressure; size it as before). Second, **pinning**: before **JDK 24**, a virtual thread blocking inside a `synchronized` block **pinned** its carrier thread, and several drivers used `synchronized` around I/O. **JEP 491 (JDK 24)** removed that limitation; on JDK 21 use drivers that replaced `synchronized` with `ReentrantLock` (pgjdbc 42.6+, recent Connector/J and Oracle drivers).

What changes in practice:

- You can drop reactive drivers (R2DBC) purely for scalability reasons in many services (Q848); plain JDBC plus virtual threads gives similar concurrency with simpler code.

© TechTribers. For personal use only. Do not redistribute.

- **ThreadLocal-heavy** code (some ORMs, MDC logging) still works, but one virtual thread per task means per-task ThreadLocals, so avoid caching expensive objects in them.
- Make sure `connectionTimeout` is sensible: thousands of virtual threads can queue for connections, and you want them to fail fast under overload rather than pile up.
- Semaphores in front of slow downstreams are a cleaner limit than pool exhaustion when one query type is expensive.

Java

```
// Spring Boot 3.2+ on JDK 21+: run request handling on virtual threads
// application.properties
//   spring.threads.virtual.enabled=true

// Plain Java: one virtual thread per task, pool as the real limit
try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<Report>> futures = customerIds.stream()
            .map(id -> exec.submit(() -> loadReport(ds, id)))   // blocking JDBC is fine
            .toList();
    for (Future<Report> f : futures) publish(f.get());
}

// JDK 21-23: find pinning caused by synchronized blocks around I/O
//   -Djdk.tracePinnedThreads=full          (removed in JDK 24, no longer needed)
//   JFR event jdk.VirtualThreadPinned
```

⚠️ **Interview trap**

Assuming virtual threads make the database faster or allow a pool of 1,000 connections. The database's parallelism is unchanged; only the Java side stops wasting threads while waiting.

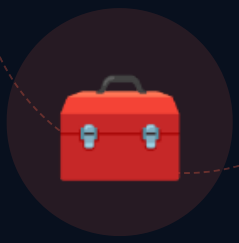
© TechTribers. For personal use only. Do not redistribute.

Section 06

Errors, Security, Testing and the JDBC Ecosystem

SQLException in depth, SQLWarning, securing database access, testing with real databases, what JdbcTemplate and JdbcClient add, choosing between JDBC, JPA, jOOQ and MyBatis, R2DBC versus virtual threads, and a reusable JDBC helper.

Questions 842–849 · 8 questions



Q842 JD-043

• Advanced

How is `SQLException` structured (SQLState, vendor code, chaining, transient vs non-transient)?

⚡ Quick answer

Every `SQLException` carries a **message**, a portable 5-character **SQLState** (class in the first two characters: `23` integrity violation, `40` transaction rollback, `08` connection problem, `42` syntax or access error), a **vendor error code** (`getErrorCode()`), a cause chain, and a **chain of further exceptions** (`getNextException()` , or iterate the exception itself). JDBC 4 adds a **subclass hierarchy**: `SQLTransientException` (retry may succeed: timeouts, rollbacks), `SQLNonTransientException` (fix the code or data: constraint violations, syntax errors) and `SQLRecoverableException` (reconnect, then retry).

Not every driver maps to the JDBC 4 subclasses consistently (pgjdbc mostly throws `PSQLException` , a plain `SQLException` subclass, and you check `getSQLState()`), so robust code checks the SQLState as well as the type.

This is exactly what Spring's `SQLExceptionTranslator` does: it maps SQLStates and vendor codes to the `DataAccessException` hierarchy (`DuplicateKeyException` , `CannotAcquireLockException` , `QueryTimeoutException` , `TransientDataAccessException` ...), which are unchecked and database-independent.

Logging advice: always log SQLState and vendor code alongside the message; they're what you search the database documentation for. Don't log bound parameter values by default (they may contain personal data).

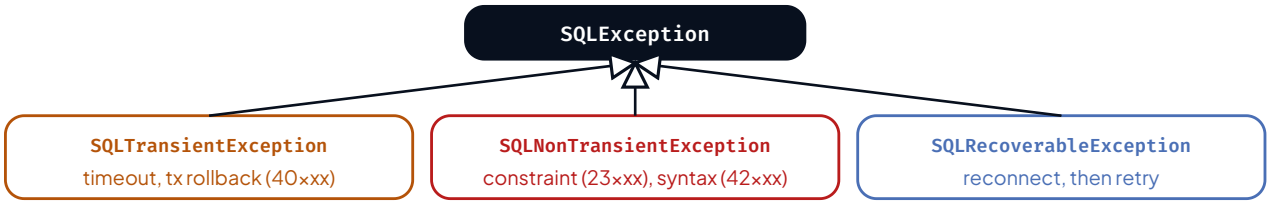
Java

```
try {
    insertCustomer(con, c);
} catch (SQLIntegrityConstraintViolationException e) {      // SQLState 23xxx
    throw new DuplicateCustomerException(c.email(), e);
} catch (SQLTransientException e) {                          // timeout, deadlock, ...
    throw new RetryableDatabaseException(e);
} catch (SQLException e) {
    for (Throwable t : e) {                                  // SQLException is Iterable
        if (t instanceof SQLException s) {
```


© TechTribers. For personal use only. Do not redistribute.

```
            log.error("SQLState={} vendor={} msg={}",
                    s.getSQLState(), s.getErrorCode(), s.getMessage());
        }
    }
    throw e;
}

// Useful SQLStates
// 23505  unique violation (PostgreSQL)      23000 + error 1062  duplicate (MySQL)
// 23503  foreign key violation              40001  serialization failure / deadlock
// 40P01  deadlock (PostgreSQL)              57014  query cancelled (timeout)
// 08001/08006  connection failure           42P01  undefined table
```



Also: BatchUpdateException, SQLWarning (not thrown), SQLTimeoutException, SQLIntegrityConstraintViolationException

`getSQLState(): portable 5 chars · getErrorCode(): vendor code · getNextException() / iterate for the chain`

Drivers don't map subclasses consistently: check the SQLState too (pgjdbc throws PSQLException).

> 💡 **Pro tip**
> Use database constraints (unique, foreign key, check) as the final guard, and translate their SQLStates into domain errors ("email already registered"). A pre-check query alone is a race condition.

⚠️ **Q843** JD-044 • Intermediate

What is `SQLWarning`, and when should you read it?

> ⚡ **Quick answer**
> An `SQLWarning` is a **non-fatal** message the driver or database attaches to a `Connection`, `Statement` or `ResultSet` instead of throwing: data truncation (`DataTruncation`), a requested result-set type or concurrency that was **downgraded**, a deprecated feature, or `RAISE NOTICE` output from a PostgreSQL function. Warnings are **not** thrown; you have to call `getWarnings()` and walk the chain with `getNextWarning()`. They're cleared on each new execution, so read them immediately after the call you care about.

When it's worth reading warnings:

- When you request non-default result-set types, concurrency or holdability (drivers report downgrades this way).
- When calling database functions that emit notices you rely on (migration scripts, maintenance procedures).

© TechTribers. For personal use only. Do not redistribute.

- When MySQL is in a non-strict SQL mode: inserting a too-long string or invalid date can produce a warning and **silently truncated or zeroed data** instead of an error. The fix there is strict mode (`STRICT_TRANS_TABLES` , default since MySQL 5.7), not reading warnings.

Spring's `JdbcTemplate` can be configured to log warnings (`ignoreWarnings=true` by default, which logs them at debug) or throw them as `SQLWarningException` .

```java
try (PreparedStatement ps = con.prepareStatement(sql,
        ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE)) {
    logWarnings("prepare", con.getWarnings());           // downgraded type/concurrency?
    try (ResultSet rs = ps.executeQuery()) {
        logWarnings("execute", ps.getWarnings());          // e.g. RAISE NOTICE output
        // ...
    }
}

static void logWarnings(String where, SQLWarning w) {
    for (; w != null; w = w.getNextWarning()) {
        log.warn("{}: SQLState={} {}", where, w.getSQLState(), w.getMessage());
    }
}
```

🎯 **Likely follow-up**

Does `DataTruncation` always come as a warning? On reads, usually a warning; on writes, many drivers throw it as an exception, because data would otherwise be lost.

Q844 JD-045 • Advanced

How do you secure JDBC access in production?

⚡ **Quick answer**

Use **least privilege** (an application role that can only run the DML it needs, a separate role for migrations, no superuser), keep **credentials out of code and URLs** (a secret manager, short-lived credentials such as IAM authentication or Vault dynamic secrets), **encrypt in transit** with TLS and **verify the server certificate** (`sslmode=verify-full` , `sslMode=VERIFY_IDENTITY`), **bind every value** and allow-list identifiers, don't log SQL parameters, and restrict network access to the database.

Points interviewers expect:

- `sslmode=require` **isn't enough**: it encrypts but doesn't verify the server, so a man-in-the-middle can still present any certificate. Use `verify-full` (PostgreSQL) or `VERIFY_IDENTITY` (MySQL).
- **Rotation**: pools hold connections for `maxLifetime` , so rotated passwords take effect gradually; dynamic-secret setups must keep `maxLifetime` shorter than the credential TTL.
- **Logs and errors**: SQL logging with parameter values, or exception messages containing row data, leak personal information. Mask or disable in production.

© TechTribers. For personal use only. Do not redistribute.

- **Row-level security** (PostgreSQL RLS, Oracle VPD) can enforce tenant isolation in the database itself, as a second line of defence behind application checks.
- **Dependencies**: keep drivers patched; drivers have had CVEs (for example pgjdbc's class-instantiation issue via connection properties, CVE-2022-21724).

Terminal

```
# PostgreSQL: TLS with server verification, credentials from the environment
spring.datasource.url=jdbc:postgresql://db.internal:5432/orders?sslmode=verify-full
spring.datasource.hikari.data-source-properties.sslrootcert=/etc/ssl/rds-ca.pem
    ?sslmode=verify-full&sslrootcert=/etc/ssl/rds-ca.pem
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}

-- Least-privilege roles (PostgreSQL)
CREATE ROLE orders_app LOGIN;
GRANT CONNECT ON DATABASE orders TO orders_app;
GRANT USAGE ON SCHEMA app TO orders_app;
GRANT SELECT, INSERT, UPDATE ON app.orders, app.order_line TO orders_app;
-- no DELETE, no DDL; migrations run as a separate role (orders_migrator)
ALTER ROLE orders_app SET statement_timeout = '30s';
```

⚠️ **Interview trap**

Running the application as the schema owner. One injection or bug can then `DROP TABLE`, and every query bypasses grants that would have contained the damage.

🧪 **Q845** JD-046 • Intermediate

How do you test JDBC code reliably (Testcontainers vs H2)?

⚡ **Quick answer**

Test SQL against the **same database engine and version** you run in production, using **Testcontainers** to start it in Docker for the test run. In-memory **H2** (even in compatibility mode) differs in SQL dialect, locking, isolation, types, error codes and time-zone handling, so H2-green tests can hide production bugs, especially for `ON CONFLICT`, JSON columns, window functions and locking. Keep tests isolated by **rolling back** each test's transaction or truncating tables, and apply the real **migrations** (Flyway/Liquibase) to the container.

Making it fast enough for every build:

- **One container per test class or per JVM** (static field, or Testcontainers' singleton pattern), not per test.
- **Reuse** containers locally (`testcontainers.reuse.enable=true`).
- In Spring Boot 3.1+, `@ServiceConnection` wires the container's URL into the `DataSource` automatically.
- Use a transaction per test with rollback for fast isolation, but also have some tests that **commit**, because commit-time behaviour (deferred constraints, triggers, isolation conflicts) only shows up there.

© TechTribers. For personal use only. Do not redistribute.

What to test at this level: mappers (every column, nulls, time zones), constraint translation, locking and concurrency behaviour (two connections racing), and query plans for critical queries (`EXPLAIN` assertions, if you need them). Pure business logic belongs in unit tests without a database.

Java

```java
@Testcontainers
class OrderRepositoryTest {

    @Container
    static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:17-alpine");

    static DataSource ds;

    @BeforeAll
    static void migrate() {
        HikariConfig cfg = new HikariConfig();
        cfg.setJdbcUrl(pg.getJdbcUrl());
        cfg.setUsername(pg.getUsername());
        cfg.setPassword(pg.getPassword());
        ds = new HikariDataSource(cfg);
        Flyway.configure().dataSource(ds).load().migrate();   // real schema
    }

    @Test
    void duplicateEmailIsRejected() throws SQLException {
        var repo = new CustomerRepository(ds);
        repo.insert(new NewCustomer("asha@example.com"));
        assertThrows(DuplicateCustomerException.class,
                () -> repo.insert(new NewCustomer("asha@example.com")));
    }
}
```

⚠️ **Interview trap**

Testing concurrency logic with one connection. Optimistic locking, `SKIP LOCKED` and deadlock retries only make sense with at least two connections running in parallel.

Q846 JD-047 • Intermediate

What do Spring's `JdbcTemplate` and `JdbcClient` remove compared with raw JDBC?

⚡ **Quick answer**

They remove the boilerplate and the mistakes: getting and releasing connections (bound to Spring transactions), creating and closing statements and result sets, binding parameters, iterating rows, and translating `SQLException` into Spring's unchecked `DataAccessException` hierarchy. You write the SQL and a row mapper. `JdbcClient` (Spring 6.1) is a fluent API on top with named or positional parameters and direct mapping to records; `NamedParameterJdbcTemplate` adds `:name` parameters and expands collections for `IN` .

© TechTribers. For personal use only. Do not redistribute.

What stays your responsibility: writing efficient SQL, transaction boundaries (`@Transactional`), batching for bulk work (`JdbcTemplate.batchUpdate`), fetch size for large reads, and avoiding N+1.

Choosing between them in Spring Boot 3.2+:

- `JdbcClient` for new code: readable, fluent, handles records and named parameters.
- `JdbcTemplate` when you need batch APIs, `ResultSetExtractor` s, stored-procedure support (`SimpleJdbcCall`) or lower-level callbacks; `JdbcClient` is built on it.
- `SimpleJdbcInsert` for inserts with generated keys without writing the SQL.

They're a good fit for read-heavy services, reporting, and teams who want SQL to stay visible.

Java

```java
// Raw JDBC: ~20 lines for one query (see Q820). Spring 6.1 JdbcClient:
record Customer(long id, String name, String email) { }

List<Customer> inCity = jdbcClient
        .sql("SELECT id, name, email FROM customer WHERE city = :city ORDER BY name")
        .param("city", "Pune")
        .query(Customer.class)                  // maps columns to record components
        .list();

Optional<Customer> one = jdbcClient
        .sql("SELECT id, name, email FROM customer WHERE id = ?")
        .param(42L)
        .query(Customer.class)
        .optional();

int updated = jdbcClient
        .sql("UPDATE customer SET email = :email WHERE id = :id")
        .param("email", newEmail)
        .param("id", 42L)
        .update();

// Collections expand for IN automatically
List<Customer> some = jdbcClient
        .sql("SELECT id, name, email FROM customer WHERE id IN (:ids)")
        .param("ids", List.of(1L, 2L, 3L))
        .query(Customer.class).list();
```

> 💡 **Pro tip**
> Spring Data JDBC (different from JdbcTemplate) adds repositories and aggregate mapping without JPA's persistence context, dirty checking or lazy loading. It's a middle ground worth knowing.

© TechTribers. For personal use only. Do not redistribute.

Q847 JD-048 • Advanced

JDBC vs JPA vs jOOQ vs MyBatis: when do you choose which?

> ⚡ Quick answer
>
> **Plain JDBC / JdbcClient**: full control, SQL visible, minimal magic, more code; good for simple services, reporting and performance-critical paths. **JPA/Hibernate**: maps object graphs, tracks changes (dirty checking), caches, and generates SQL; productive for rich domain models with mostly CRUD, but you must understand fetching, flushing and the persistence context. **jOOQ**: type-safe SQL built in Java from the schema, so you keep SQL's full power with compile-time checks. **MyBatis**: SQL in XML or annotations mapped to objects; popular for hand-tuned SQL in large enterprise codebases.

A practical decision guide:

- **Write-heavy domain logic with aggregates and invariants** → JPA (or Spring Data JDBC), using DTO projections for reads.
- **Read-heavy, report-like, complex SQL** (window functions, CTEs, vendor features) → jOOQ or JdbcClient.
- **Bulk processing** → JDBC batches or `COPY`; an ORM adds overhead per entity.
- **Mixed** is normal: many teams use JPA for commands and JdbcClient/jOOQ for queries in the same service (a light CQRS split), sharing one transaction manager and `DataSource`.

Whatever you pick runs on JDBC underneath, so pool sizing, transactions, isolation, fetch size and batching knowledge applies to all of them.

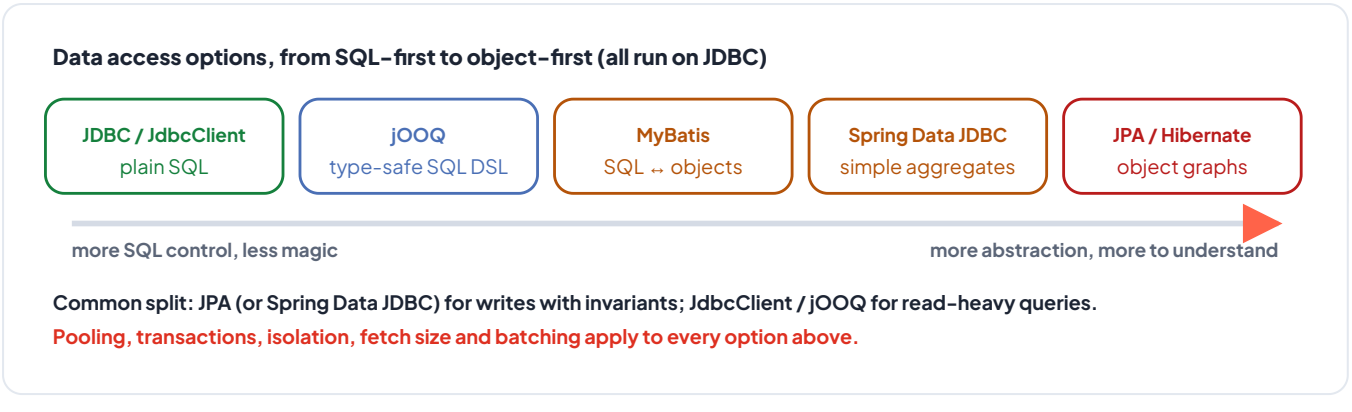
Java

```java
// JPA: navigate the object graph, SQL is generated
Order o = em.find(Order.class, id);
o.getLines().add(new OrderLine(sku, 2));      // flushed on commit (dirty checking)

// jOOQ: SQL with compile-time checking from generated classes
List<OrderSummary> top = ctx
        .select(ORDERS.ID, ORDERS.TOTAL, CUSTOMER.NAME)
        .from(ORDERS).join(CUSTOMER).on(CUSTOMER.ID.eq(ORDERS.CUSTOMER_ID))
        .where(ORDERS.CREATED_AT.gt(OffsetDateTime.now().minusDays(7)))
        .orderBy(ORDERS.TOTAL.desc())
        .limit(10)
        .fetchInto(OrderSummary.class);

// JdbcClient: plain SQL string, explicit mapping
List<OrderSummary> top2 = jdbcClient.sql("""
        SELECT o.id, o.total, c.name
        FROM orders o JOIN customer c ON c.id = o.customer_id
        WHERE o.created_at > now() - interval '7 days'
        ORDER BY o.total DESC LIMIT 10""")
        .query(OrderSummary.class).list();
```


© TechTribers. For personal use only. Do not redistribute.



⚠️ **Interview trap**

Picking JPA and then writing all queries as native SQL strings. You pay the ORM's complexity without its benefits; either embrace entities for writes or choose a SQL-first tool.

🔀 **Q848** JD-049 • Advanced

What is R2DBC, and do virtual threads make it unnecessary?

⚡ **Quick answer**

R2DBC is a reactive, non-blocking database API (Reactive Streams based) with its own drivers, used with Project Reactor in Spring WebFlux and Spring Data R2DBC. It lets a few threads handle many concurrent queries without blocking. **Virtual threads** (Java 21) give similar scalability with **blocking JDBC** and ordinary imperative code, and JDBC has the larger ecosystem (JPA, jOOQ, mature drivers, tooling). For most new services that need scalability, virtual threads plus JDBC is now the simpler choice; R2DBC remains reasonable in an already fully reactive stack or for streaming back-pressure end to end.

Trade-offs to state:

- **R2DBC limits**: no JPA (Hibernate Reactive is a separate project with its own API), fewer drivers and features, transactions and debugging are harder, and stack traces are less useful.
- **Virtual-thread limits**: the connection pool is still the real concurrency limit; long blocking calls still hold connections; very high fan-out streaming may still suit reactive back-pressure better.
- **Neither** makes the database handle more concurrent queries. Both just stop the application side from wasting threads.

A good interview answer mentions that Spring Boot 3.2+ (`spring.threads.virtual.enabled=true`) made the virtual-thread route a one-line change for MVC applications.

Java

```
// R2DBC with Spring Data: everything is a Publisher
Flux<Order> recent = databaseClient
        .sql("SELECT id, total FROM orders WHERE customer_id = :id")
        .bind("id", customerId)
        .map((row, meta) -> new Order(row.get("id", Long.class),
                                      row.get("total", BigDecimal.class)))
```


© TechTribers. For personal use only. Do not redistribute.

```
        .all();

// JDBC on a virtual thread: same concurrency, plain code
@GetMapping("/customers/{id}/orders")
List<Order> recent(@PathVariable long id) {            // runs on a virtual thread
    return jdbcClient.sql("SELECT id, total FROM orders WHERE customer_id = ?")
            .param(id).query(Order.class).list();
}
```

🎯 **Likely follow-up**

Can you mix them? Technically yes, but it means two data-access stacks, two transaction models and two pools. Pick one per service.

🛠️ **Q849** `JD-050` • Advanced

Coding: write a small reusable JDBC helper with query, update and transaction support.

⚡ **Quick answer**

Wrap a `DataSource` in a class that offers `query(sql, mapper, params)`, `queryOne`, `update(sql, params)` and `inTransaction(work)`. Inside a transaction, every call must use the **same connection**, so keep the current connection in a `ScopedValue` or `ThreadLocal` and fall back to borrowing one per call otherwise. Use try-with-resources everywhere, bind parameters with `setObject`, map rows with a `RowMapper<T>`, and roll back on any throwable.

Design points to talk through:

- **Connection binding**: the `ThreadLocal` is what makes `update` calls inside `inTransaction` share one connection; it's the same idea as Spring's `TransactionSynchronizationManager`. On Java 25 a `ScopedValue` is a cleaner fit, because the binding can't leak beyond the transaction's scope.
- **Joining**: a nested `inTransaction` joins the outer one (like `Propagation.REQUIRED`); a production helper would also support `REQUIRES_NEW` and savepoints.
- `prepare` **and resource safety**: if binding throws, the statement leaks; a stricter version closes it in a catch block.
- **What's missing on purpose**: batching, generated keys, fetch size, timeouts and exception translation, all easy to add with the same structure.

Writing this once is a good exercise; in a Spring application, use `JdbcClient` and `@Transactional`, which do the same with far more edge cases handled.

Java

```
public final class Jdbc {
    @FunctionalInterface
    public interface RowMapper<T> { T map(ResultSet rs) throws SQLException; }

    @FunctionalInterface
```


© TechTribers. For personal use only. Do not redistribute.

```
    public interface Work<T> { T run() throws SQLException; }

    private final DataSource ds;
    private final ThreadLocal<Connection> current = new ThreadLocal<>();

    public Jdbc(DataSource ds) { this.ds = ds; }

    public <T> List<T> query(String sql, RowMapper<T> mapper, Object... params)
            throws SQLException {
        return withConnection(con -> {
            try (PreparedStatement ps = prepare(con, sql, params);
                 ResultSet rs = ps.executeQuery()) {
                List<T> out = new ArrayList<>();
                while (rs.next()) out.add(mapper.map(rs));
                return out;
            }
        });
    }

    public <T> Optional<T> queryOne(String sql, RowMapper<T> mapper, Object... params)
            throws SQLException {
        List<T> rows = query(sql, mapper, params);
        if (rows.size() > 1) throw new SQLException("expected one row, got " + rows.size());
        return rows.stream().findFirst();
    }

    public int update(String sql, Object... params) throws SQLException {
        return withConnection(con -> {
            try (PreparedStatement ps = prepare(con, sql, params)) {
                return ps.executeUpdate();
            }
        });
    }

    public <T> T inTransaction(Work<T> work) throws SQLException {
        if (current.get() != null) return work.run();        // join the outer tx
        try (Connection con = ds.getConnection()) {
            con.setAutoCommit(false);
            current.set(con);
            try {
                T result = work.run();
                con.commit();
                return result;
            } catch (Throwable t) {
                con.rollback();
                throw t;
            } finally {
                current.remove();
                con.setAutoCommit(true);
            }
        }
    }

    private interface ConFn<T> { T apply(Connection con) throws SQLException; }

    private <T> T withConnection(ConFn<T> fn) throws SQLException {
        Connection tx = current.get();
```


© TechTribers. For personal use only. Do not redistribute.

```
        if (tx != null) return fn.apply(tx);                // inside inTransaction
        try (Connection con = ds.getConnection()) {
            return fn.apply(con);
        }
    }

    private static PreparedStatement prepare(Connection con, String sql, Object... params)
            throws SQLException {
        PreparedStatement ps = con.prepareStatement(sql);
        for (int i = 0; i < params.length; i++) ps.setObject(i + 1, params[i]);
        return ps;
    }
}

// Usage
Jdbc jdbc = new Jdbc(dataSource);
long id = jdbc.inTransaction(() -> {
    jdbc.update("UPDATE account SET balance = balance - ? WHERE id = ?", amount, from);
    jdbc.update("UPDATE account SET balance = balance + ? WHERE id = ?", amount, to);
    return from;
});
```

⚠️ **Interview trap**

Using a `static ThreadLocal` shared by several `Jdbc` instances pointing at different databases. A transaction on database A would then be joined by calls meant for database B. Keep the binding per `DataSource`, as here.

© TechTribers. For personal use only. Do not redistribute.

Series progress

849 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 18 starts at Q850.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52
Part 12	Q480–Q529	50
Part 13	Q530–Q579	50
Part 14	Q580–Q679	100
Part 15	Q680–Q739	60
Part 16	Q740–Q799	60
Part 17	Q800–Q849	50

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.